

A Constant-Rate Compiler for MPC over Noisy Networks

Ran Gelles¹, Carmit Hazay¹, Manuj Mukherjee², Jaspal Singh^{3,4}, Arun Yerragudipati³, and Vassilis Zikas⁴

¹ Bar-Ilan University

² Indraprastha Institute of Information Technology, Delhi

³ Purdue University

⁴ Georgia Institute of Technology

{Carmit.Hazay, ran.gelles}@biu.ac.il, manuj@iitd.ac.in, sing1361@purdue.edu, ysarun1999@gmail.com, vzikas@gatech.edu,

Abstract. The study of efficient multi-party computation (MPC) has been a central focus in the cryptographic literature, producing a wide range of innovative techniques that have substantially improved the practicality of MPC in real-world applications. However, the vast majority of this work assumes reliable communication channels and neglects the impact of network-level noise—a fundamental characteristic of modern communication systems. Although classical error-correcting codes can be used to address such noise, they often introduce significant overhead, potentially offsetting the efficiency gains provided by advanced cryptographic methods. To the best of our knowledge, existing efforts to circumvent this limitation rely on alternative techniques restricted to the two-party setting, with approaches that do not naturally generalize to the multi-party case.

We initiate the study of information-theoretic MPC over noisy networks for more than two parties. We construct an MPC protocol that is secure against a semi-honest majority with an optimal communication overhead in circuit size: it computes any circuit $\mathcal{C}$ with communication complexity proportional to its size, i.e., $O(|\mathcal{C}|)$. Our protocol departs from the classical MPC methodology by introducing a novel multi-party *interactive-coding* that remedies channel noise through a new rewinding technique that also maintains the properties required for secure computation. The ideas of our interactive coding can be applied also to other MPC scenarios to eliminate channel noise. In fact, our treatment uncovers (and resolves) a subtle flaw in the constant-rate 2PC construction by Gelles et al. [SCN '18] over noisy channels.

1 Introduction

Secure Multi-Party Computation (MPC) enables a group of n parties, each holding a private input $x_1, \dots, x_n$, to jointly compute a public function $f(x_1, \dots, x_n)$ while ensuring that any subset of corrupted parties learns nothing more than their own inputs and the function's expected output. Beginning with the work due to [Yao82], MPC has garnered significant attention in cryptographic literature. Over the years, a substantial body of work has been dedicated to exploring MPC under different adversarial models, computational assumptions, and efficiency metrics. This body of research has explored both theoretical and practical aspects of MPC, with a particular emphasis on enhancing security, minimizing the assumptions required, and reducing computational overhead to make the protocols more feasible for real-world applications. In this work, we explore dishonest majority MPC in the presence of passive (semi-honest) adversaries¹ for Boolean circuits.

Despite extensive study of efficient MPC in different settings, relatively little research has been devoted to understanding its feasibility over noisy communication channels. In real-world networks, communication may be subject to environmental interference, hardware malfunctions, or even adversarial noise. The ability to design MPC protocols that remain correct and private under such

¹ A semi-honest adversary follows the protocol correctly but tries to infer additional private information from the protocol messages of corrupt parties.

noisy conditions would enable direct deployment over unreliable networks (e.g., UDP-based communication) without requiring additional error correction mechanisms.

A standard approach to mitigating channel noise is to employ error-correcting codes (ECC); Such codes allow transmitting a message over the *binary symmetric channel with error ε* (BSC_ε) that flips each transmitted bit with some fixed probability ε , so that the receiver is able to decode it with high probability which depends on the code length. Then, a natural way to make a given MPC protocol π work when executed over BSC_ε , is to encode each protocol message using ECC separately. While this transformation preserves the security guarantees of π , it introduces a significant communication overhead, specifically logarithmic in the number of communicated messages, which potentially undermines the efficiency benefits of the original protocol.

Let us demonstrate the above trade-off in a concrete example: consider the case where each message sent during the GMW protocol [GMW87] for a circuit C with $2^{-\kappa}$ statistical security is encoded with a standard ECC of length ℓ ; this means every message is decoded correctly except with probability $2^{-\Theta(\ell)}$. Note that the entire protocol fails if even a single message gets corrupted. Thus, to ensure that the failure event remains with probability at most $2^{-\kappa}$, we must have $\ell = \Omega(\kappa + \log |C|)$.² As a result, the total communication overhead of this approach is $O_{n,\varepsilon}(|C|(\kappa + \log |C|))$, where $O_{n,\varepsilon}(\cdot)$ follows the standard notation of asymptotics, treating n and ε as constants, thus hiding them in the $O(\cdot)$ notations.

The above limitation appears inherent in any approach that encodes each message independently. This leads us to the following question:

Is it possible to design an MPC protocol over BSC_ε channels with communication complexity $O_{n,\varepsilon}(|C| + \kappa)$ for any circuit C ?

We'll refer to such MPC protocols with communication complexity linear in the circuit size i.e. $O_{n,\varepsilon}(|C|)$ as *circuit-scalable*. In this work, we give an affirmative answer to the above question for semi-honest majorities in the strongest possible manner, i.e., we design a circuit-scalable MPC protocol with information-theoretic security over noisy channels that does not assume any setup.

At first glance, the above statement might appear contradictory. Indeed, it is known that dishonest (or even semi-honest) majority MPC is impossible without a setup, such as Oblivious Transfer (OT), for achieving information-theoretic security. Nevertheless, noisy channels themselves can be leveraged as a cryptographic setup, even for information-theoretic security. For instance, [CK88b, Cr 97, HIKN08] construct a statistically secure OT protocol from BSC_ε channels, whereas OT over noiseless channels generally requires additional cryptographic assumptions and only achieves computational security [CSSW06]. This makes the study of MPC over noisy channels even more interesting: not only does it extend MPC to a more general communication setting, but it also offers a potential resource that can be leveraged to overcome limitations in traditional communication settings.

As mentioned above, using a separate ECC on each message will not lead to our desired result. Instead, we use the method of *interactive coding* (IC) [Sch92, Sch93, Gel17], in order to obtain noise-resilience while maintaining a constant-rate. Multiparty IC schemes transform any n -protocol π over noiseless channels into a functionally equivalent protocol π' over noisy BSC channels (or even more adversarial noise models) [RS94, JKL15, LV15, HS16, CGH19, ABE⁺19, CLPS20, GKR21, GKR22]. Most of these schemes are *constant-rate* in the sense that the communication complexity of the

² For circuits with large width (i.e., many gates evaluated in parallel at a given depth), this ECC-based variant of GMW could be slightly optimized by jointly encoding multiple message bits exchanged in the same round into a single ECC codeword. However, for general circuits, this choice of ℓ is asymptotically minimal.

resulting protocol grows linearly with that of the original, i.e., $\text{CC}(\pi') = O(\text{CC}(\pi))$, where $\text{CC}(\pi)$ denotes the communication complexity in bits of the protocol π . Unfortunately, none of the existing multi-party IC schemes are suitable for our goals, as they are not *privacy-preserving*, and cannot be directly used to construct MPC protocols over noisy channels. We therefore develop a novel IC scheme especially designed to be both rate- and privacy-preserving.

At a high level, our approach leverages many pre-computed OTs from BSC_ε to construct a circuit-scalable semi-honest-majority MPC protocol over noisy channels, which is then coded via our novel IC scheme.

We note in passing that one might naïvely presume that such an interactive-coding approach is unnecessary and one can just construct a protocol π that is secure against an active (malicious) adversary over reliable channels, which would then directly be semi-honest secure over simple noisy channels. Indeed, it is tempting to think that the BSC_ε -induced “noise” could be captured as a form of adversarial behavior in the (malicious) noiseless setting. This presumption is wrong: The key issue is that security against a malicious adversary does not necessarily guarantee that a deviating corrupted party receives the correct output. In contrast, in the semi-honest setting, even when the channel is noisy, the protocol must ensure that all honest parties still receive the correct output, implying a non-trivial transition from a maliciously secure protocol over reliable channels to a semi-honest secure protocol over noisy channels.

1.1 Our Contributions

We present the first n -party statistically secure MPC protocol for arbitrary Boolean circuits $\mathcal{C}$ over BSC_ε channels, achieving communication complexity $O_{n,\varepsilon}(|\mathcal{C}|)$. Our construction is secure against static, passive adversaries corrupting up to $n - 1$ parties. In particular, we prove the following theorem, stated informally,

Theorem 1 (Main theorem, Informal). *For any circuit $\mathcal{C} : \{0,1\}^m{}^n \rightarrow \{0,1\}^{m'}$ and any inputs $x_i \in \{0,1\}^m$, there exists a n -party protocol that evaluates the circuit $\mathcal{C}(x_1, \dots, x_n)$ over BSC_ε channels that is statistically secure against semi-honest adversaries corrupting $\leq n - 1$ parties. Moreover, the communication complexity of this protocol is $O_{n,\varepsilon}(|\mathcal{C}| + \kappa)$, where κ is a statistical security parameter.*

Our key technical contribution is a new multi-party interactive coding scheme, presented in Section 3. This scheme transforms the GMW protocol π [GMW87] (a semi-honest secure MPC protocol over noiseless channels) into an equivalent secure protocol π' , ensuring that it maintains the same input-output behavior over noisy channels as π does over noiseless ones. Crucially, our transformation is circuit-scalable, implying an MPC protocol with communication complexity $O(|\mathcal{C}|)$. While our original scheme has $O(|\mathcal{C}|)$ round complexity, in Section 6 we discuss how the round complexity of our compilation can be reduced to $O(\kappa \cdot \text{depth}(\mathcal{C}))$ while preserving both asymptotic communication complexity and security of our scheme over BSC . This can be contrasted to the classical ECC-based approach mentioned earlier, which separately encodes each message. The latter approach preserves round complexity, but incurs super-constant communication overhead in circuit size. In particular, our IC approach surpasses the naïve ECC approach for protocols with a large number of messages or when the depth of $\mathcal{C}$ is large.

Central to our contribution is the identification of key properties that IC schemes must satisfy in order to ensure privacy in addition to error correction. In particular, we analyze what fundamentally

“goes wrong” when prior IC schemes are directly applied to GMW-based secure computation. These critical properties are discussed in more detail in Section 1.2. Once these required properties are identified and formalized, we design a novel IC scheme that satisfies them. This scheme is then used on top of the GMW protocol (in particular, a variant of the GMW protocol [GMW87], instantiated with the constant-rate OT from [HIKN08]) to fortify it against channel noise while preserving its security and maintaining its asymptotic communication complexity.

We note that our technique is non-black-box in the underlying GMW protocol and is specifically tailored to its structure. Consequently, the same technique applied to other secure protocols does not necessarily preserve their privacy and correctness. Designing a generic method that applies to any MPC protocol while maintaining its communication and round complexities remains an interesting open problem for future research. In our setting, since we rely on standard GMW protocol with communication complexity $O(n^2|C|)$, the resulting MPC protocol over BSC has complexity $O_\epsilon(n^2|C|)$. Instead, a generic compiler that simultaneously preserves rate and privacy could leverage more communication-efficient MPC protocols over noiseless channels (like [FY92, CCXY18, PS21, IPS08, BCG⁺19]) to give more efficient protocols over BSC. Our work also mostly focuses on asymptotic communication complexity over noisy networks, and we leave as future work to optimize our construction for concrete protocols and practical noise regimes.

1.2 Technical Overview

In this part, we highlight the challenges of designing a secure and circuit-scalable information-theoretic MPC protocol over noisy channels and introduce key technical ideas underlying our construction. As a starting point, we first examine a natural but flawed approach that fails to achieve security in this setting. This approach serves as a template for our final protocol and assists in understanding the necessary properties of our IC scheme.

Our starting point is the multi-party coding scheme by Alon et al. [ABE⁺19], which enables the simulation of any protocol π over fully connected networks where each channel is an independent BSC. This scheme incurs only a constant overhead, completing in $O(|\pi|)$ rounds of communication between each pair of parties, and succeeds with probability $1 - 2^{-\Omega(\sqrt{n}|\pi|)}$. A natural first attempt is to apply this IC scheme to the classic n -party GMW protocol [GMW87], originally designed for noiseless channels, and execute it over noisy BSC channels. However, this direct compilation fails to yield a secure MPC protocol. In particular, two fundamental issues arise when employing the scheme of [ABE⁺19] in this context.

Issue I: Privacy violation. The scheme in [ABE⁺19] simulates π round by round unless some inconsistency is observed by the parties, e.g., due to channel noise. In such cases, the parties may rewind the simulation to a previous round of π that predates the observed inconsistency. However, applying this approach directly to the GMW protocol introduces challenges. The GMW protocol evaluates a sequence of NAND gates, each involving two calls to an oblivious transfer (OT) protocol (see Section 5.1). Naively rewinding the GMW execution may violate its security guarantees, particularly if OT invocations are repeated or reused without proper safeguards.

To illustrate this issue, consider the simpler two-party AND functionality, where Alice and Bob hold input bits x and y , respectively, and the protocol outputs random XOR shares of $x \wedge y$ to both parties. Figure 1 presents a protocol for realizing this functionality in the random OT hybrid model [Bea91]. In this protocol, the parties effectively implement a 1-out-of-2 OT by exchanging three-bit messages. Alice’s OT input messages are $(r, r + x)$, where r is a uniformly random bit and

$+$ denotes XOR. Bob's OT input is the selection bit y , and he receives the corresponding message $r' = r + (x \wedge y)$ as output. Hence, the pair (r, r') forms an XOR sharing of $x \wedge y$. This functionality is realized via the random OT primitive, which provides correlated OT triples: Alice receives random bits (r_0, r_1) , and Bob receives a random selection bit b along with the corresponding message r_b from Alice's pair.

Protocol Π_{AND}

Inputs: Alice input is a bit x ; Bob inputs a bit y .

1. Parties make a call to $\mathcal{F}_{\text{ROT}}$: Alice gets r_0, r_1 , while Bob gets b and r_b as output.
2. Bob sends $c = y + b$ to Alice, and let its received bit be c' .
3. Alice samples $r \leftarrow_{\$} \{0, 1\}$ and let $(m_0, m_1) = (r, r + x)$.
4. Alice sends $m_0 + r_{c'}, m_1 + r_{1-c'}$ to Bob, and let the received bits be (z_0, z_1) .
5. Alice outputs r and Bob outputs $z_y + r_b$.

Fig. 1: The Π_{AND} Protocol

While this protocol is correct and secure over noiseless channels, we highlight how executing this protocol multiple times with same inputs and randomness (as needed due to rewinding in IC schemes) could lead to privacy violations over BSC channels. Define $(\varepsilon_1, \varepsilon_2, \varepsilon_3) \in \{0, 1\}^3$ as indicators for the event of a bit flip in each of the three bits exchanged in the protocol over a noisy network, respectively. Let say the AND protocol is executed twice with the same inputs (x, y) and same random OT correlation $((r_0, r_1), (b, r_b))$ but with different noise indicators, say, $(0, 0, 0)$ and $(1, 0, 0)$. In these two executions, let the messages received by Bob in the protocol be (z_0, z_1) and (z'_0, z'_1) respectively. Bob can further compute the following given that it has r_b :

$$\begin{aligned} z_y + r_b &= (m_y + r_b) + r_b = m_y = r + (x \wedge y) \\ z'_{y+1} + r_b &= (m_{y+1} + r_b) = m_{y+1} = m_y + x \end{aligned}$$

In other words, Bob learns both messages $(m_0, m_1) = (r, r + x)$, one from each execution of the OT functionality. This follows because randomness r_b is used as the mask for Alice's messages m_y and m_{y+1} in the first and second OT invocations respectively. Furthermore, Bob can compute Alice's input as $(z_y + z'_{y+1}) = x$, violating the privacy of the AND functionality. A similar issue arises when executing a single NAND operation in an IC scheme compilation of GMW protocol that allows for rewinding.

To maintain the security of OT and NAND operations in the presence of communication corruption, we must ensure that any noise occurring during the execution of these operations does not leak information. The natural way to achieve this is by rewinding to the beginning of the OT protocol and re-executing it *with fresh randomness*. This necessitates considering the following simulation approach: instead of operating on a per-round basis, the simulation should proceed in chunks of c rounds each, for some constant c . If a rewind is triggered, the simulator should discard any incomplete suffix of a chunk and roll back to its beginning. For our MPC simulation, we define a chunk to correspond to the computation of a single NAND gate, which requires two pairwise OT instances. Since each OT instance consists of three rounds, a chunk encompasses $c = 6$ rounds in total. However, re-simulating the same block with fresh randomness introduces new challenges that have not been addressed by existing IC schemes, which we describe next.

Issue II: Correlation consistency. The second and more critical issue arises due to the reliance of π on correlated randomness. When parties execute an OT instance in GMW, they must use the *same* correlated randomness to ensure a valid computation. In GMW, these correlations correspond to pairwise random OT triplets, which must remain consistent across all executions. However, prior IC schemes, and in particular the one from [ABE⁺19], allows different parties to simulate the same round of π at different times. While this flexibility is harmless for deterministic protocols (or whenever the randomness of different parties is not correlated), it becomes critical in our setting, since it allows different parties to use *different* random OT triplet for computing the same OT in GMW. This means that even if the messages exchanged between the parties appear consistent (there was no channel noise), the underlying computation may be invalid and the output of the protocol renders incorrect. However, the IC scheme will not be able to detect this issue, since it does not stem from channel noise! Indeed, from the point of view of each party, the messages it sends are fully consistent to the messages it has transmitted (and the randomness it holds).

We note in passing that the two-party compiler from [GPZ18] also suffers from the above issue, requiring a modification to their underlying IC scheme. They propose using an IC scheme (due to [Hae14]) to compile the 2-party GMW protocol to give constant-rate 2PC over BSC channels. However, the IC scheme due to [Hae14] does not ensure that both parties simulate each round simultaneously, which could violate the correctness of the underlying GMW due to inconsistent correlations. This “bug” can be fixed by replacing their IC scheme with our multi-party IC scheme described below.

A new multi-party IC scheme. Towards our noise-resilient MPC, we have developed a new interactive-coding scheme that addresses both issues mentioned above. To ease the presentation, we describe how to solve each issue in isolation; combining these two approaches leads to the final coding scheme presented in Section 4.

The first coding scheme, given in Section 3.1, extends the IC scheme of [ABE⁺19] (which by itself is based on the scheme in [RS94]) so that the simulation works in chunks of c rounds each, rather than in a round-by-round fashion. One possible approach toward that goal is to simulate π round by round, rewinding to the beginning of the previous chunk whenever an inconsistency is detected. This approach is slightly more difficult to analyze because parties may be simulating different rounds and may have to rewind a different number of rounds to reach the nearest start of a chunk. Instead, we take a more rigorous approach, where the simulation takes structured chunk-size steps in a synchronized manner.

In the scheme we design, the simulation of each chunk consists of two parts: In the first part, the parties simulate c rounds of π , where each round is practically encoded via a standard error correcting code (the A2A scheme, Section A.1). In the second part, the parties exchange a *single* tree-code symbol³ that either encapsulates their view of the last c simulated rounds or contains a “rewind” instruction (denoted BK). The consistency check is then based on decoding the sequence of received tree-code symbols and contrasting them with the c bits communicated in the first part of each chunk so far. If all the transmissions seem consistent with the received tree-code symbols, the simulation progresses to the next chunk. Otherwise, during the simulation of the next chunk, the party ignores the first part and sends a BK symbol during the second part. This instructs the other parties to rewind the simulation by a single chunk.

³ A *tree code* is an online encoding technique with many applications in interactive coding; see Def. 5 and Appendix A.1 for further details.

The advantage of this structure is twofold: (a) the parties are always synchronized within a single chunk: in the first part, they all send the bits π sends during that chunk, and in the second part, they all send a tree-code symbol. Note that different parties may still simulate different chunks, but all parties are synchronized on each round *within* a chunk. (b) the analysis becomes very simple, following the same analysis performed in [RS94, Raj94], with relatively only small adaptations. See Section 3.1 for details.

Next, we deal with the requirement that all nodes use the same correlated randomness when simulating a specific chunk. Our approach modifies the “consistency check” part of the coding scheme so that a party verifies the following two properties at every round. (1) Similar to the scheme in [RS94, ABE⁺19], every sent message must correspond to the information received so far. (2) Additionally, any given chunk of π has been simulated by all the parties *exactly* at the same round of the simulation scheme. To verify property (2), each party records the round numbers used for simulating chunks that made it to the current “final” transcript (i.e., chunks that were not rewound and removed from the transcript). Then, the party processes the tree-code symbols received from each neighbor and deduces, for each simulated chunk, the round number in which that neighbor has simulated the chunk. If the simulation of all chunks seems consistent and synchronous, the party continues to simulate the next chunk. Otherwise, it rewinds.

However, it turns out that rewinding one chunk whenever an inconsistency is found might be catastrophic to the success of the coding scheme. To illustrate this issue, consider the case where at the end of some simulation round ($t = 0$) all parties have simulated up to chunk i correctly. Now suppose that in the next round ($t = 1$), there is a communication error that makes party v believe that there is an inconsistency, so it rewinds one chunk back to $i - 1$. At the same time, party w continues in its simulation to chunk $i + 1$. In the next round ($t = 2$), both parties detect inconsistencies: w detects the rewinding of v and rewinds one chunk, as it needs to re-simulate chunk $i - 1$. However, party v has just simulated chunk $i - 1$ in round $t = 2$ while party w had simulated it earlier, causing a synchronization inconsistency. If this case is not properly dealt with and v rewinds one chunk as well, it can happen that v and w will continue rewinding all the way to chunk 0, deleting any progress they have made so far.

Instead, we introduce a “hold” instruction, represented as the tree-code symbol H . When a party v detects that it needs to re-simulate some chunk and there exists a different party w that is currently farther ahead of v , then the party v *does not rewind* but rather “holds” at the same place and waits until w rewinds. When all parties become synchronized again, they all re-simulate the next chunk at the same simulation round, maintaining the required synchronization.

Adding a hold instruction complicates the parties’ behavior and this complicates the correctness analysis of the coding schemes. Fortunately, we can show that the coding scheme converges correctly with high probability. Furthermore, despite the need to “stop” the simulation until all parties rewind to the same point, we show that a single channel corruption delays the simulation by only $O(1)$ rounds, guaranteeing a constant rate for this scheme. The detailed analysis is given in Section 3.2.

MPC over noisy channels. Our proposed MPC protocol over BSC_ε based on the classic GMW protocol has the following outline:

1. In the input sharing and output reconstruction phase: use standard error-correcting codes for sending each message between parties, as specified by the GMW protocol. Using ECC of length $O_\varepsilon(\kappa)$, ensure that each received decoded message matches the corresponding sent message in the GMW protocol with $1 - \text{negl}(\kappa)$ probability.

2. For the evaluation phase: use our proposed IC scheme (from Theorem 7) with chunk size 6 (ensuring each chunk corresponds to a single NAND execution) to execute the GMW evaluation phase, which can be viewed as a protocol with random OT correlations.

The correctness of our MPC protocol reduces to the correctness of the classic GMW protocol. This is because the IC scheme guarantees that each party simulates any round of the GMW protocol in the same iteration, ensuring that the same instance of correlated randomness is used. Additionally, it is straightforward to verify that the constant-rate underlying IC scheme implies the communication cost of the MPC protocol is $O_{n,\varepsilon}(|\mathcal{C}| + \kappa)$, where $\mathcal{C}$ is the circuit computed by the MPC protocol.

The privacy proof is more involved. In Theorem 8, we note that an A2A variant of the GMW NAND execution over noisy channels, although not always correct, satisfies a notion of privacy. Specifically, there exists a simulator whose output, when given corrupt party inputs, is indistinguishable from the view of this protocol over noisy channels. We leverage this noisy A2A-NAND simulator and the security guarantee of the classic GMW NAND protocol (over noiseless channels) to construct the simulator for our proposed MPC protocol. Intuitively, the MPC simulator repeatedly invokes the noisy NAND simulator to generate views for each chunk, except for those chunks where an output gate is processed correctly by all parties. In those cases, the view is generated by the classic (noiseless) GMW NAND simulator given corrupt party inputs and the ideal MPC protocol output. This description glosses over a few details, including how the simulator generates a view that’s indistinguishable from the tree-code symbols exchanged in each chunk. A more detailed discussion of the simulator and the privacy proof can be found in Section 5.

1.3 Additional Related Works

Secure Computation over Noisy Channels. Ishai et al. [IKO⁺11] proposed a protocol for two parties to evaluate a circuit over noisy channels with communication complexity $O(|\mathcal{C}|) + \text{poly}(\kappa \cdot \text{depth}(\mathcal{C}))$. Their protocol works in the malicious setting and achieves statistical security using the IPS compiler [IPS08]. Following that, Gelles et al. [GPZ18] were able to reduce the communication complexity to $O(|\mathcal{C}| + \kappa)$ by using an interactive coding scheme in the semi-honest setting. In the case of malicious adversaries in the presence of adversarial channels, an information-theoretic result by Chung et al. [CPT13] showed that there exists no interactive coding scheme that has an error rate better than the naive approach of encoding each message with an ECC. Moreover, the authors showed a positive result in the computational setting, highlighting a tradeoff between the error rate and the communication complexity. A similar result was shown for semi-honest adversaries in the presence of adversarial channels by Gelles et al. [GSW14].

Beyond BSC channels, a line of research has focused designing MPC protocols over other noisy channels including *elastic channels* [DFMS04, Wul09, KMS16]. In an elastic channel, an adversarial receiver can increase the reception reliability unbeknown to the sender, but the sender cannot modify the channel’s error probability. The work of Khurana et al. [KMS16] and Cascudo et al. [CDLR16] fully characterizes the parameter for elastic channels that enable secure computation. We believe our techniques can also be applied to design circuit-scalable MPC protocols over elastic channels for some parameter range, and we leave this as future work.

Interactive Coding Schemes [Gel17]. The seminal work of Schulman [Sch92, Sch93] introduced interactive coding schemes and showed the first schemes that overcome random and adversarial noise in the two-parties setting. In particular, the work [Sch93] defines the notion of tree-codes

and proves that tree-codes with good parameters exist. This result was extended by Rajagopalan and Schulman [RS94] to the multi-party setting, showing a scheme with rate $\Omega_\varepsilon((d+1)^{-1})$ where d is the maximal degree of a node in the network. Alon et al. [ABE⁺19] extended this scheme to obtain a constant rate for a complete graph. The works [BEGH16, GK19, MG24] prove a lower bound on the coding overhead for certain topologies. For the case of adversarial noise, multi-party coding schemes resilient to some fraction of the noise were presented under different noise models and communication assumptions settings [JKL15, LV15, CGH19, CLPS20, GKR21, GKR22].

2 Preliminaries

2.1 Notations

Here, we describe the basic notations used throughout this work. For any positive integer $k \in \mathbb{N}$, let $[k]$ represent the set $\{1, 2, \dots, k\}$. The notation $\leftarrow_{\$} S$ and $\leftarrow \mathcal{D}$ signifies sampling at random from set S and from the distribution $\mathcal{D}$ respectively. $\text{bernoulli}(p)$ represents the standard Bernoulli distribution with parameter $p \in [0, 1]$: it outputs bit 1 with probability p and 0 with probability $1 - p$. For a matrix M , M_i is its i^{th} row and M^i is its i^{th} column. For any set $S \subseteq [n]$ for any vector a , we use the shorthand notation $a_S = \{a_i \mid i \in S\}$ whenever convenient. For algorithms with for loops or summation over any variable i , we omit its range of values if it is clear from the context. For functions f, g , the notation $f(n, m) = O_m(g(n))$ indicates that the function f grows at most at the order of g when m is treated as a constant. To simplify notation, whenever it is clear from the context, we might write $O(\cdot)$ instead of $O_{n,\varepsilon}(\cdot)$, etc.

Protocols with correlations. In this work, we will specifically consider multi-party protocols where parties also take as input a vector of correlations. The set of all correlations can be specified by a set $\mathbb{C}$, such that each element of the set is a vector of size n (i.e., the number of parties) - corresponding to components of the correlations that each party uses. If a protocol uses t instances of $\mathbb{C}$ correlations, then we represent the vector of all correlations as $\text{corr} \in \mathbb{C}^t$. We can view the vector of vectors corr as a matrix of size $t \times n$. Hence, $\text{corr} = (\text{corr}^1, \dots, \text{corr}^n)$, where each corr^i represents the t correlation shares that are used by Party P_i during the execution of the protocol.

Let $\mathbb{C}_{\text{ROT}}$ represent the set of all random OT (ROT) correlations between two parties. $\mathbb{C}_{\text{ROT}}$ contains all elements of the form $((r_0, r_1), (b, r_b))$, where $r_0, r_1, b \in \{0, 1\}$, where the sender's and receiver's correlation components are (r_0, r_1) and (b, r_b) respectively. In our work, we particularly assume n -party correlations, which contain $\mathbb{C}_{\text{ROT}}$ correlations for every pair of parties, where the party with the lower index acts as the sender and the party with the higher index acts as the receiver.

We particularly consider protocols with correlations that use a new correlation instance every t rounds in the protocol, where t is a constant. We present a formal definition in Appendix A that encapsulates the correctness of a protocol with respect to input correlations.

2.2 MPC Preliminaries

We consider n -party protocols that evaluate n -input functions. A function $f : \{\{0, 1\}^m\}^n \rightarrow \{\{0, 1\}^{m'}\}^n$ takes n inputs $x = (x_1, \dots, x_n) \in \{\{0, 1\}^m\}^n$, where each party P_i inputs x_i , and the protocol outputs a m' length bit-string to each party. Let $f(x) = (f_1(x), \dots, f_n(x))$ represent the n outputs.

In each round of any protocol, each party sends a single bit to every other party, including itself. We consider a fully connected network with point-to-point channels between every two parties. To simplify the protocol description, we assume that the network has self-loops. Messages sent by a party to itself are not considered to be sent over the network. We assume private channels, i.e., the adversary interacting with the protocol, can only see all messages a party receives by corrupting it. A party's transcript is defined as the set of received messages in the protocol.

In the context of general MPC we will consider functions that can be represented as boolean circuits consisting of only NAND gates (computing negation of AND of input bits), which is a complete basis for boolean circuits. A gate in the circuit is called an output gate if its output wire corresponds to one of the outputs of the entire circuit $\mathcal{C}$. The size of the circuit $|\mathcal{C}|$ precisely corresponds to the number of NAND gates in the circuit.

MPC security over noiseless channels. We consider standard simulation-based statistical semi-honest security for n parties based on the real-world/ideal-world paradigm of [Can00]. Parties participate in an execution of π over noiseless channels in the presence of an adversary Adv who corrupts a set C of parties. Define the view of a set of parties C as their inputs, any internal randomness used by the parties, the correlated randomness of all parties in C , followed by the messages transcript of each party in the set C . More formally, for statistical security parameter κ , let View_π be the view defined as:

$$\text{View}_\pi(C; x, \kappa) = \{x_C, r_C, \text{corr}^C, T_C\}$$

where T_C, corr^C, r_C is the set of messages received, correlated randomness, and internal randomness, respectively, of all parties $i \in C$ in the execution of π over noiseless channels. Also let $\text{Output}_\pi(x, \kappa)$ represent the output of all parties in the execution. We also define the augmented view of the corrupt parties (View'_π) to include the transcript of the honest parties, alongside the output of $\text{View}_\pi(C; x, \kappa)$.

A real-world protocol π is said to be secure against semi-honest adversaries if whatever can be learned from the real-world adversary can be learned from its inputs and outputs alone. That is, a protocol π is secure if there exists an efficient simulator Sim which, given the security parameter, inputs and outputs of the corrupt parties, can generate the view of the real-world adversary Adv .

Definition 1 (Semi-Honest Security over Noiseless Channels). *An n -party protocol π for computing a function f is said to be statistically secure against semi-honest adversaries over noiseless channels if, for any $C \subset [n]$, for every real-world adversary Adv corrupting a set C of parties, there exists an efficient simulator Sim such that, for all inputs x and statistical security parameter κ , the following holds:*

$$\{\text{Sim}(C; 1^\kappa, x_C, f_C(x)), f(x)\} \approx_{\exp(-\kappa)} \{\text{View}_\pi(C; 1^\kappa, x), \text{Output}_\pi(1^\kappa, x)\}$$

Further, we also consider a stronger security notion called *semi-honest security with channel transparency*, where the simulator can generate the augmented view of the corrupted parties, such that the joint view of the augmented view and the protocol output are indistinguishable from the simulator output and the ideal output. We say that a semi-honest secure MPC protocol π for computing a function represented by a boolean circuit $\mathcal{C}$ is *circuit-scalable* if the communication complexity of π is $O_n(|\mathcal{C}|)$, i.e., the communication overhead is linear in the circuit size.

Oblivious Transfer. The Oblivious Transfer (OT) [Rab81, EGL82] functionality $\mathcal{F}_{\text{OT}}$ is a 2-party functionality between two parties, a sender and a receiver. The input of the sender consists of two bits (x_0, x_1) , and the receiver's input is a single choice bit b . The output of the functionality $\mathcal{F}_{\text{OT}}((x_0, x_1), b)$ is $(\perp, x_b)$, where the receiver learns the bit x_b (while learning nothing about x_{1-b}) and the sender learns nothing (about the value b). Random Oblivious Transfer functionality ($\mathcal{F}_{\text{ROT}}$) is similar to the OT functionality, but the parties provide no inputs, and the protocol outputs a random instance of OT $(x_0, x_1), (b, x_b)$, where $x_0, x_1, b \leftarrow_{\$} \{0, 1\}$.

In this work, we design protocols in $\mathcal{F}_{\text{ROT}}$ -hybrid model, and later instantiate this ROT functionality with a statistically secure random OT protocol. By the composition theorem in [Can00], the protocol remains secure after instantiating the ROT ideal functionality with a secure protocol if the original protocol was secure in ROT-hybrid functionality.

Protocols over noisy channels. For protocols over unreliable, i.e., noisy channels, we assume that any point-to-point channel between any two parties is unreliable. In particular, in this work, we consider channels that stochastically flip every transmitted bit with a constant probability ε , independent of other bits. This corresponds to the versatile extension of the well-known binary symmetric channel with crossover probability ε , i.e., BSC_{ε} . The notion of a protocol must be extended to the above noisy communication model. Note that in this case, the parties may have inconsistent views of the transmitted and received messages, depending on the noise. When the protocol π is executed over BSC channels, we represent the corresponding view and output with functions $\text{View}_{\pi}^{\text{BSC}}$ and $\text{Output}_{\pi}^{\text{BSC}}$, where BSC is in the superscript.

Definition 2 (Semi-Honest Security over BSC_{ε} Channels). *An n -party protocol π for computing a function f is said to be statistically secure against semi-honest adversaries over BSC_{ε} if, for any $C \subset [n]$, for every real-world adversary Adv corrupting a set C of parties, there exists an efficient simulator Sim^{BSC} such that, for all inputs x and statistical security parameter κ , the following holds:*

$$\{\text{Sim}^{\text{BSC}}(C; 1^{\kappa}, x_C, f_C(x)), f(x)\} \approx_{\exp(-\kappa)} \{\text{View}_{\pi}^{\text{BSC}}(C; 1^{\kappa}, x), \text{Output}_{\pi}^{\text{BSC}}(1^{\kappa}, x)\}$$

Random OT over BSC. In this work, we design our generic MPC protocol in the $\mathcal{F}_{\text{ROT}}$ -hybrid model over the BSC channels in Theorem 10. Our MPC protocol for computing a circuit $\mathcal{C}$ over private inputs invokes the ROT functionality $O_{n,\varepsilon}(|\mathcal{C}|)$ times. We can instantiate this 2-party functionality with the construction from [HIKN08], which achieves semi-honest security over BSC channels. Using their technique, one can obtain $O(\ell)$ ROT correlations between any two parties with communication complexity $O_{\varepsilon}(\ell)$ bits.

Theorem 2 ([HIKN08, Theorem 9]). *For any constant $\varepsilon \in (0, 1/2)$ and any ℓ , there exists a two-party protocol $\pi^{\text{OT}^{\ell}}$ that securely realizes $\mathcal{F}_{\text{ROT}}$ functionality over BSC_{ε} channels. The protocol is statistically secure against semi-honest parties with error $2^{-\Omega(\ell)}$, and it has communication complexity $O_{\varepsilon}(\ell)$.*

The construction of $\pi^{\text{OT}^{\ell}}$ follows the “MPC-in-the-head” paradigm, where the two parties emulate the MPC virtual protocol by playing the roles of multiple parties. In our case, $\pi^{\text{OT}^{\ell}}$ is instantiated with the BGW protocol [BOGW88] as the multi-party protocol in the OT hybrid model. For the OTs, a protocol based on [CK88a] is used, which has perfect privacy but its correctness fails with a constant probability. This entire construction assumes that the BSC channels are private, i.e., the view of the adversary does not contain the received messages or the transcript of the honest parties.

2.3 Coding Schemes Preliminaries

Further Notation: For an event A , we let $\mathbf{1}_A$ be the indicator variable that takes the value 1 when the event A occurs or otherwise takes the value 0. Given any finite set (alphabet) Σ , the set Σ^* denotes the set of all possible strings of finite length over the alphabet Σ . Given any $x \in \Sigma^*$, we let $\text{len}(x) = |x|$ denote the length of x . For any vector (or string) x , the notation $x[i]$ denotes the i -th component of x and $x[i, j]$ with $1 \leq i < j \leq |x|$ denotes the respective sub-vector or sub-string. Note that $x[i]$ can itself be a string or a vector. For X , a variable of our algorithm, we denote with $X(r)$ the value of X at the *end* of round r of the algorithm.

Network: We consider a clique of n parties $\{p_i\}_{i \in [n]}$ where the party p_i holds an input x_i . Each pair of parties (p_i, p_j) are connected by a bidirectional communication link. The network is synchronous, and at any round r any party p_i can communicate a single symbol (from some fixed channel-alphabet Σ_C) on each link connected to it. The party p_i may communicate a different symbol on each link.

The parties wish to run a *fully-utilized* protocol π in order to compute some (possibly randomized) function $f(x_1, \dots, x_n)$ of their private inputs. A fully utilized protocol proceeds in rounds, where at every round, p_i sends one symbol from Σ to each of its neighbors. The symbol sent to p_j depends on the private input x_i and all the symbols that p_i has received from *all* its neighbors until the start of that round. Let us use the following notation to describe the above. For any round $r \geq 1$, let $m_{\rightarrow i}(r) \triangleq (m_{1 \rightarrow i}(r), \dots, m_{n \rightarrow i}(r))$ be all the messages (symbols of Σ_C) received by p_i until round r (inclusive). We assume $m_{j \rightarrow i}(0)$ and $m_{i \rightarrow i}(r)$ to be empty for any i, j, r . We similarly define $m_{i \rightarrow}(r) \triangleq (m_{i \rightarrow 1}(r), \dots, m_{i \rightarrow n}(r))$ as all the symbols sent by p_i up to round r , inclusive. Thus, in a fully utilized protocol, party p_i uses $\pi(x_i, m_{\rightarrow i}(r))$ to compute the symbol it wishes to send to each p_j in round $r+1$, records it in $m_{i \rightarrow}$, receives a symbol from each p_j , records it in $m_{\rightarrow i}$, and so on.

The total number of rounds of π is referred to as its *round complexity*, and is denoted by $|\pi|$. We shall refer to $m_{i \rightarrow}(|\pi|)$ and $m_{\rightarrow i}(|\pi|)$ respectively as the *sent and received transcripts* of p_i .

Coding Scheme: Now, consider the situation where the communication links between each pair of parties are noisy. In particular, each link is a binary symmetric channel BSC_ϵ . To simulate π , the parties run a new protocol $\tilde{\pi}$, such that each party p_i can deterministically reconstruct the transcript $m_{i \rightarrow}(|\pi|)$ and $m_{\rightarrow i}(|\pi|)$ based on x_i and the sent and received transcripts of p_i for $\tilde{\pi}$. We say that the simulation is successful if the reconstruction is without errors at all parties p_i .

The following well-known result says that a successful simulation is always possible with a constant blow-up in the number of rounds.

Theorem 3 (Theorem 1 in [ABE⁺19]). *There exists an n -party simulation $\tilde{\pi}$ for every fully-utilized protocol π on a clique, with $|\tilde{\pi}| = O(|\pi|)$ and probability of error $2^{-\Omega(|\pi|\sqrt{n})}$.*

The simulation $\tilde{\pi}$ of Theorem 3 involves a *layered coding* approach, with the standard Rajagopalan-Schulman (RS) scheme of [RS94] being the outer code, while the A2A scheme of [ABE⁺19] being the inner code. The A2A scheme solves the following bit-exchange task. Assume that each party p_i holds $n-1$ bits $b_i = (b_{i,1}, \dots, b_{i,n})$, where $b_{i,j}$ needs to be communicated to p_j . To accomplish this, the parties are allowed to run a fully-utilized protocol on the clique with noisy links and reconstruct the $n-1$ bits they need to learn based on their respective sent and received transcripts. The bit-exchange task is successful if all the parties p_i are able to correctly recover the bits $b_{j,i}$ for $j \in [n]$. The A2A scheme is an $O(1)$ -round protocol that accomplishes the broadcast task with a vanishing probability of error, as summarized in the theorem below.

Theorem 4 (Theorem 4 of [ABE⁺19]). *The A2A scheme with round complexity $O(1)$ successfully performs the bit-exchange task, except with probability $2^{-\Omega(\sqrt{n})}$.*

The RS scheme allows simulating with $O(1)$ overhead, any protocol π over arbitrary network assuming BSC links with sufficiently small flip probability ε . The A2A provides this sufficiently small flipping overhead over a clique, and together, the concatenated scheme achieves simulation of any π with $O(1)$ overhead with high probability over arbitrary BSC_ε channels with any $\varepsilon < 1/2$.

A more detailed discussion of the RS scheme and a formal description of the A2A primitive is provided in Appendix A.1.

In our work, we also assume standard error-correcting codes ECC as implied by the work of Shannon [Sha48]. This scheme allows us to encode any message with an overhead of $O(\kappa)$ and ensure that decoding fails over BSC with probability $\leq 2^{-\kappa}$. A formal description of this coding scheme is presented in Appendix A.1.

Rate of IC schemes. An interactive coding scheme C takes a deterministic protocol π defined over noiseless channels and transforms it into a randomized protocol π' over noisy (BSC) channels such that the output of π' is equal to the output of π with very high probability and constant rate of communication. A constant rate of a scheme means that $\frac{\text{cc}(\pi)}{\text{cc}(\pi')} = O(1)$ for any π on any input. Note that this rate differs from the rate of MPC protocols, which measures the communication complexity of the MPC protocol with respect to the size of the boolean circuit it is computing.

3 Two New Multi-Party Interactive Coding Scheme Solving Two Issues

To design our MPC protocol over BSC channels, we require a multiparty IC scheme that is (i) *chunked* i.e., it works on continuous chunks of c rounds instead of a round-by-round fashion; and (ii) the IC scheme simulates any round of the underlying noiseless protocol in exactly the same round for all the parties. In Section 3.1, we introduce a constant-rate IC scheme called ChunkRS that has the first mentioned property, and in Subsection 3.2, we introduce a constant-rate IC scheme that satisfies the second property of simultaneous simulation. Finally, in Subsection 4, we describe our final constant-rate IC scheme that satisfied both these properties simultaneously.

3.1 Chunked Coding Scheme on a Clique

In this part, we present a chunked version of the RS scheme on a clique network, where each link is a BSC_ε channel, i.e., $\Sigma_C = \{0, 1\}$ and each bit is independently flipped with probability $\varepsilon < 1/2$.

While the ordinary RS scheme in [RS94] simulates the noiseless protocol π in a round by round fashion, our new ChunkRS simulation directly simulates a chunk of $c = O(1)$ consecutive rounds at a time, where the integer $c \geq 1$ is a parameter of the simulation that can be chosen arbitrarily.

Let π be the noiseless protocol we wish to simulate, and assume that any round beyond $|\pi|$ simply instruct the parties to exchange zeros with each other. Define $k \triangleq \frac{2|\pi|}{c}$ to be the number iterations we need to execute in order to complete the simulation of π (this corresponds to [RS94] which runs for $2|\pi|$ iterations, where every iteration simulates a single bit). The scheme ChunkRS (Algorithm 1) runs for k iterations (chunks) of the loop in line 2, which we now describe. Each iteration has two phases. The first phase involves c uncoded bit exchanges via the A2A scheme; this is performed as part of the `SimChunk()` Procedure, depicted in Algorithm 2. The second phase of communication involves exchanging one *tree code* symbol, encoded into bits, via the A2A scheme (lines 21 and 26 in

Algorithm 1 The ChunkRS scheme (actions taken by party p_i)

input: A protocol π with zero-padding after $|\pi|$ transmissions; An input x_i ; A tree-code $\mathcal{T} : S^* \rightarrow \Sigma^*$ with distance $\alpha = \frac{1}{2}$; A chunk size c .

- 1: **initialize** $T_i \leftarrow \emptyset, R_i \leftarrow \emptyset$
- 2: **for** iteration $l = 1$ **to** $\frac{2|\pi|}{c}$ **do**
- 3: **for** all $j \in [n]$ **do**
- 4: Let $\rho_{i,j}$ be all symbols received from p_j so far
- 5: Set $w_{i,j} = \mathcal{T}^{-1}(\rho_{i,j})$ be the tree-code decoding of the received symbols
- 6: $\tilde{z}_{i,j} \leftarrow \text{ParseChunk}(w_{i,j})$
- 7: $z_{i,j} \leftarrow \text{ParseChunk}(R_{i,j})$
- 8: $\sigma_{i,j} \leftarrow \text{ParseChunk}(T_{i,j})$
- 9: **end for**
- 10: flag $\leftarrow 1$
- 11: **for** all $j \neq i$ **do** ▷ Checking the consistency of the transcript
- 12: **if** $z_{i,j}$ is not a prefix of $\tilde{z}_{i,j}$ **then**
- 13: flag $\leftarrow 0$
- 14: **end if**
- 15: **end for**
- 16:
- 17: **if** flag = 0 **then** ▷ Error detected, simulate arbitrary information and send rewind tree-code symbol
- 18: $u_i, v_i \leftarrow \text{SimChunk}(\text{flag}, \tilde{z}_i, z_i, \sigma_i)$
- 19: $T_{i,j} \leftarrow T_{i,j} \circ \text{BK}, \quad \forall j$
- 20: $R_{i,j} \leftarrow R_{i,j} \circ \text{BK}, \quad \forall j$
- 21: Send the l -th symbol of $\mathcal{T}(T_{i,j})$ to p_j for all $j \neq i$ via A2A scheme
- 22: **else** (flag = 1) ▷ No error detected, simulate next chunk and send the respective tree-code symbol
- 23: $u_i, v_i \leftarrow \text{SimChunk}(\text{flag}, \tilde{z}_i, z_i, \sigma_i)$
- 24: $T_{i,j} \leftarrow T_{i,j} \circ u_{i,j}, \quad \forall j$
- 25: $R_{i,j} \leftarrow R_{i,j} \circ v_{i,j}, \quad \forall j$
- 26: Send the l -th symbol of $\mathcal{T}(T_{i,j})$ to p_j for all j via A2A scheme
- 27: **end if**
- 28: **end for**
- 29: **return** $\text{ParseChunk}(T_{i,j}), \text{ParseChunk}(R_{i,j})$ truncated to $\frac{|\pi|}{c}$ chunks, for all j .

Algorithm 2 The SimChunk procedure for chunk l (actions taken by party p_i)

input: $\text{flag}, \tilde{z}_i, z_i, \sigma_i$

1: **initialize:** $u_{i,j}, v_{i,j} \leftarrow \emptyset, \forall j \in [n]$

2: **if** $\text{flag} = 1$ **then**

3: **for** $q = 1$ **to** c **do**

4: **for all** j **do**

5: $u_{i,j}[q] \leftarrow$ Next bit p_i should send to p_j based on $\pi(x_i, z_i \circ v_i)$

6: **end for**

7: Using A2A, send $u_{i,j}[q]$ to all p_j 's and receive their respective bit $\text{temp}_{i,j}[q]$.

8: **for all** j **do**

9: $r \leftarrow |z_{i,j}|$

10: **if** $|\tilde{z}_{i,j}| > r$ **then** $\triangleright$ Use previously sent bit, if p_j is ahead

11: $v_{i,j}[q] \leftarrow$ the q -th bit of the string $\tilde{z}_{i,j}[r+1]$

12: **else** $\triangleright p_i, p_j$ simulate the same chunk, record the last simulated bit

13: $v_{i,j} \leftarrow v_{i,j} \circ \text{temp}_{i,j}[q]$

14: **end if**

15: **end for**

16: **end for**

17: **else** $\triangleright \text{flag} = 0$, send arbitrary information

18: Execute A2A for c consecutive times, with the all-zero string as input.

19: $u_{i,j} \leftarrow 0^c, \quad \forall j \neq i$

20: $v_{i,j} \leftarrow 0^c, \quad \forall j \neq i$

21: **end if**

22: **return** u_i, v_i

Algorithm 1). The tree code⁴ used in the second phase is $\mathcal{T} : S^* \rightarrow \Sigma^*$ with distance $\alpha = \frac{1}{2}$, where the input alphabet $S \triangleq \{0, 1\}^c \cup \{\text{BK}\}$. In a sense, the **SimChunk** procedure simulates a chunk, and then the tree-code verifies the simulation and instructs the parties whether the simulation succeeded (when the tree-code symbol matches the simulation in **SimChunk**) or when some discrepancy is detected (then, the next tree-code symbol will be BK, namely “back”, instructing a rewind).

Detailed Description of the chunked RS scheme. Every party p_i keeps a record of $n - 1$ pairs of strings $(T_{i,j}, R_{i,j}) \in (\{0, 1\}^c \cup \{\text{BK}\})^*$, for each $j \neq i$ it communicates with. To ease notations, p_i will also hold similar variables for itself (i.e., $T_{i,i}, R_{i,i}$) assumed to be all zeros and effectively ignored by the algorithm. We can now denote, for any $i \in [n]$, $T_i = (T_{i,1}, \dots, T_{i,n})$, and $R_i = (R_{i,1}, \dots, R_{i,n})$. We will use a similar notation for all other variables of the algorithm: $v_{i,j}, u_{i,j}, z_{i,j}, w_{i,j}$, etc.

The strings T_i and R_i record the sent and received transcripts of p_i respectively. The party p_i will also keep a record of all the tree coded symbols it has received so far from each other party, namely, the symbols received from p_j are kept at $\rho_{i,j}$.

To proceed with the description of **ChunkRS**, we define a parse operation, **ParseChunk()**, similar to the parse operation in [RS94], yet we work in chunks of c bits rather than in bits. The parse operation takes as input a string over the alphabet S , parse it symbol by symbol consecutively and record each symbol. However, once a BK symbol is encountered, instead of recording it, the procedure deletes the last recorded symbol (if any). If the operation runs out of symbols to delete while encountering a BK symbol, then **ParseChunk** returns the null string.

We continue describing **ChunkRS** from the view of party p_i . Suppose we are at the beginning of the l -th iteration (chunk), for some $l \in [k]$. At first, party p_i decodes the $(l - 1)$ -length string of tree-code symbols it has received so far from p_j (for each $j \in [n]$), and obtains the decoded string $w_{i,j} \in S^{l-1}$. Next, p_i parses $w_{i,j}$ to obtain $\tilde{z}_{i,j}$, which is the estimated sent messages of p_j for the current round. To verify that the computation so far makes sense, $\tilde{z}_{i,j}$ must be contrasted with the parsed recorded sent and received transcript, $z_{i,j}, \sigma_{i,j}$, generated from $R_{i,j}, T_{i,j}$ respectively (lines 7 and 8). We comment here that the lengths of all the string $z_{i,j}, \sigma_{i,j}$ are same for all j 's, since the BK symbol is always simultaneously added to all $T_{i,j}, R_{i,j}$ for all $j \neq i$ in Algorithm 1.

Next, party p_i checks whether its actions so far are consistent with the protocol π . To do so, it check whether its recorded transcript $z_{i,j}$ agrees with the prefix of $\tilde{z}_{i,j}$ of same length as $z_{i,j}$ (line 12). If the recorded transcript is consistent with the current estimation(s) of received symbols, then the consistency check is passed, denoted by assigning the value 1 to the consistency-flag variable **flag**; otherwise, if the consistency check fails even for a single p_j , we assign **flag** = 0. We note here that as a byproduct of the above consistency check, Algorithm 1 ensures that z_i and σ_i are also always consistent.

Following the consistency check, p_i begins the first phase of communication for chunk l (Algorithm 2). In this first communication phase, p_i simulates the next chunk of π by executing the **SimChunk** procedure. If the consistency check in this chunk failed (line 18), p_i already knows that it needs to delete its incorrect transcript. So, in this phase, it simply sends c successive zeroes to all other parties using the **A2A** scheme, irrespective of what it received. Otherwise, it will simulate the next chunk according to π (line 23). Recall that when more than $\frac{|\pi|}{c}$ chunks have been successfully simulated, we zero pad π for any chunk beyond the first $\frac{|\pi|}{c}$.

A key point here is what should p_i consider as the received bits from other parties, in order to know what next bit to send next when simulating the next chunk of π . Suppose $|z_{i,j}| = |\tilde{z}_{i,j}|$, i.e., p_i

⁴ See Appendix A.1 for the definition and description of tree codes.

has decoded in this iteration exactly the same amount of (parsed) symbols as it has recorded so far. Then, p_i believes everything is synchronized between itself and p_j , and assumes that the interaction with p_j (performed in line 13 in algorithm 2) is the correct continuation of the simulation. However, if the received symbols indicate a longer transcript than recorded, $|z_{i,j}| < |\tilde{z}_{i,j}|$ then p_i will base the simulation on the estimated chunk received from p_j , i.e., on $\tilde{z}_{i,j}[|z_{i,j}| + 1]$ (line 11 in algorithm 2).

Upon completing the simulation of the current chunk, party p_i records the simulated chunk(s) $u_{i,j}, v_{i,j} \in \{0, 1\}^c$, for all $j \neq i$; these are the c bits p_i sent and received from p_j , respectively. The derailed simulation of a specific chunk is given in Algorithm 2.

To proceed with the second phase of communication in iteration (chunk) l , party p_i appends BK to $T_{i,j}, R_{i,j}$ for any p_j that did not pass the consistency check (lines 19 and 20). If it did pass, then it appends $T_{i,j}, R_{i,j}$ respectively with $u_{i,j}, v_{i,j}$ available from the simulation step (lines 24 and 25). Next, p_i performs the tree code encodings $\mathcal{T}(T_{i,j})$, and wishes to send the last (i.e., the l -th) symbol of $\mathcal{T}(T_{i,j})$ to p_j . To do so, it encodes that symbol from Σ into $\log |\Sigma|$ bits and exchanges them one-by-one using the A2A scheme. This concludes the second phase of communication the l -th chunked round.

After running $k = \frac{2|\pi|}{c}$ successive chunked rounds, p_i outputs the parsed versions of $T_{i,j}, R_{i,j}$, for all parties p_j , as its respective sent and received transcripts, truncated to $\frac{|\pi|}{c}$ chunks, i.e., to $|\pi|$ bits.

Proposed ChunkRS satisfies the following theorem, which is formally proven in Appendix B.

Theorem 5. *Consider a clique of n parties whose links are noisy BSC_ε links, for some $\varepsilon < 1/2$. Then, the ChunkRS scheme with parameter $c > 1$ (Algorithm 1), successfully simulates any fully-utilized protocol π , in chunks of size c , in $O_\varepsilon(|\pi|)$ rounds, with a probability of error $2^{-\Omega_\varepsilon(c(|\pi|\sqrt{n}))}$.*

3.2 Multi-Party Coding Scheme with Simultaneous Simulation

In this section we construct a variant of the RS scheme [RS94] over noisy cliques, with an additional property: when a party p_i simulates some round of π , all other parties simulate the same round of π . This is critical when π assumes round-based shared randomness.

We use similar notations to the sections above. Our goal is stated as the following theorem.⁵

Theorem 6. *Given any set of n parties with pairwise noisy bidirectional communication links, there exists a simulation protocol $\tilde{\pi}$ which ensures correct and simultaneous simulation of every (noiseless) protocol π in $O(|\pi|)$ rounds, except with probability $2^{-\Omega(|\pi|\sqrt{n})}$ over the random noise.*

3.3 Overview of Simultaneous Simulation

The simulation $\tilde{\pi}$ runs on a quaternary alphabet $S = \{0, 1, \text{BK}, \text{H}\}$. To describe the simulation, we shall make use of a tree code $\mathcal{T} : S^* \rightarrow \Sigma^*$ of parameter $\alpha = \frac{1}{2}$, see Appendix A.1 for a formal definition of a tree code. To simplify notations, we will assume that the channel alphabet is also Σ , i.e., in any round of the simulation parties exchange a single tree-code symbol on each link. Received symbols may be corrupted in a manner that is equivalent to looking on the $\log |\Sigma|$ bits describing a single symbol, and flipping each bit independently with probability ε .

Fix π and let $R = |\pi|$. We now give an overview of the simulation, formally depicted in Algorithm 3 for the actions taken by party i . The simulation consists of $3R$ iterations,⁶ divided into two

⁵ We consider only sufficiently long protocols, i.e., $|\pi| = \omega(n)$, since otherwise we could simply simulate every round with A2A to achieve a similar result.

⁶ In order to distinguish rounds of π from rounds of the simulation $\tilde{\pi}$ we will call the former *rounds* and the later *iterations*.

Algorithm 3 Multi-party Coding Scheme with simultaneous simulation (actions taken by p_i)

inputs: a. A protocol π with zero-padding after $R \triangleq |\pi|$ transmissions; b. An input x_i ; c. A tree-code $\mathcal{T} : S^* \rightarrow \Sigma^*$ with distance $\alpha = \frac{1}{2}$.

- 1: **initialize** $s_{i,j}, w_{i,j} \leftarrow \emptyset, \forall j \neq i$
- 2: **for** $r = 1$ to $2R$ **do**
- 3: $\hat{w}_{i,j} \leftarrow$ the decoding of the tree-coded string (of length $r - 1$) received from p_j so far
- 4: $(\sigma_{i,j}, \hat{z}_{i,j}, z_{i,j}) \leftarrow \text{PARSE}(s_{i,j}, \hat{w}_{i,j}, w_{i,j})$
- 5: $\text{flag} \leftarrow \text{ConsCheck}(\{\sigma_{i,j}, \hat{z}_{i,j}, z_{i,j} \mid j \neq i\})$
- 6: $l \leftarrow |\sigma_{i,j}|$ for some $j \neq i$ ▷ The choice of j doesn't affect the algorithm
- 7: $\text{Ind}_i \leftarrow \text{GenIndex}(s_{i,j})$ for some $j \neq i$ ▷ The choice of j doesn't affect the algorithm
- 8: $\text{Ind}_{i,j} \leftarrow \text{GenIndex}(\hat{w}_{i,j})$ ▷ Simulate the next round
- 9: **if** $\text{flag} = 1$ **and** $\text{Ind}_i[r - 1] = l$ **and** for all $j \neq i$, $\text{Ind}_{i,j}[r - 1] = l$ **then**
- 10: **for** all $j \neq i$ (in parallel) **do**
- 11: $w_{i,j}[r - 1] \leftarrow \text{Info}(\hat{z}_{i,j}[l])$ ▷ "Record" used received symbol
- 12: $\text{temp}_j \leftarrow \text{Info}(z_{i,j}) \circ w_{i,j}[r - 1]$
- 13: $(s_{i,1}[r], \dots, s_{i,n}[r]) \leftarrow \pi(x_i, \{\text{temp}_j\}_{j \in [n]})$ ▷ next round of π
- 14: Send $\mathcal{T}(s_{i,j})[r]$ to p_j using A2A
- 15: **end for** ▷ Some neighbour is ahead, hold
- 16: **else if** $\text{flag} = 1$ **and** $\text{Ind}_i[r - 1] = l$ **and** for all $j \neq i$, $\text{Ind}_{i,j}[r - 1] \geq l$ **then**
- 17: **for** all $j \neq i$ (in parallel) **do**
- 18: $w_{i,j}[r - 1] \leftarrow \text{H}$
- 19: $s_{i,j}[r] \leftarrow \text{H}$
- 20: Send $\mathcal{T}(s_{i,j})[r]$ to p_j using A2A
- 21: **end for**
- 22: **else** ▷ Inconsistency found, rewind
- 23: **for** all $j \neq i$ (in parallel) **do**
- 24: $w_{i,j}[r - 1] \leftarrow \text{BK}$
- 25: $s_{i,j}[r] \leftarrow \text{BK}$
- 26: Send $\mathcal{T}(s_{i,j})[r]$ to p_j using A2A
- 27: **end for**
- 28: **end if**
- 29: **end for**
- 30: **for** $r = 1$ to R **do** ▷ Dummy Phase, send more redundancy
- 31: Send $\mathcal{T}(s_{i,j} \circ 0^r)[2R + r]$ to p_j using A2A
- 32: **end for**
- 33: $\hat{w}_{i,j} \leftarrow$ the decoding of the tree-coded string (of length $3R$) received from p_j so far
- 34: For all $j \neq i$: $(\sigma_{i,j}, \hat{z}_{i,j}, z_{i,j}) \leftarrow \text{PARSE}(s_{i,j}, \hat{w}_{i,j}, w_{i,j})$
- 35: **return** $(\sigma_{i,j}, z_{i,j})_{j \neq i}$ with all strings truncated to the first R indices

phases. The first phase, referred to as the *information phase*, consists of $2R$ iterations of actual information exchange between the parties. This is then followed by a *dummy phase* of R iterations. The purpose of the dummy phase is to ensure that the parties correctly decode the tree-code symbols of the information phase.

For any $j \in [n]$, party p_i hold a pair of strings $s_{i,j}, w_{i,j} \in S^*$, recording the symbols p_i have sent and received from p_j . In more detail, $s_{i,j}$ is the transcript of sent symbols, and $w_{i,j}[1, r-1]$ is the recorded set of symbols that p_i believes p_j has sent; these are the symbols that were used to generate the sent symbol $s_{i,j}[1, r]$. In addition, p_i maintains the string $\hat{w}_{i,j}$, recording the symbols received from p_j . Note that these may differ from $w_{i,j}$ due to errors. In a sense, $w_{i,j}$ is p_i 's estimation of the symbols p_j sent, based on the possibly corrupted received $\hat{w}_{i,j}$.

Very roughly, the actions taken by p_i in iteration r of the simulation is as follows. For every $j \neq i$, party p_i decodes all the tree-code symbols received from p_j so far (i.e., $r-1$ such symbols) and obtains the string $\hat{w}_{i,j} \in S^{r-1}$. Then, it checks whether (1) the simulation so far is consistent with π and (2) the simulation is simultaneous. This is performed via the `ConsCheck()` procedure stated in Algorithm 6, based on p_i 's knowledge about symbols sent to and received from other parties, i.e., $s_{i,j}, w_{i,j}$ for $j \neq i$, and the freshly decoded $\hat{w}_{i,j}$. Note that the “recorded symbols” string $w_{i,j}$ always lags one symbol behind $s_{i,j}, \hat{w}_{i,j}$, (i.e., it is of length $r-2$); for all p_i computations that involve the received transcript from p_j , the first $r-2$ symbols are taken from $w_{i,j}$, while the $(r-1)$ -th symbol is taken from the freshly decoded $\hat{w}_{i,j}$.

If `ConsCheck` indicates that the simulation is not consistent with π or that it is not simultaneous, p_i assigns $s_{i,j}[r] = \text{BK}$ for all p_j , which indicates it wishes to remove the last simulated round from the parties' recorded transcripts. Additionally, p_i sets $w_{i,j}[r-1] = \text{BK}$ for all p_j s to delete the record of the symbols received from p_j s in the previous iteration. This is because a failed consistency check supposedly indicates that either the previous round was not simultaneously simulated, or the recorded received symbols from some p_j in the previous iteration was incorrect. If the simulation is consistent and simultaneous, but p_i believes that at least one of its neighbours have simulated at least one extra round, then p_i decides to hold its current position (i.e., it does not continue to simulating the next round of π). Technically, it assigns $s_{i,j}[r] = w_{i,j}[r-1] = \text{H}$ for all p_j s. Finally, if p_i believes that the simulation so far has been simultaneous and consistent, and every neighbour has simulated exactly the same number of rounds, then it decides to simulate the next round of π . To that end, it sets $s_{i,j}[r]$ according to the next symbol to be sent according to π . It also assigns $w_{i,j}(r-1) = \hat{w}_{i,j}(r-1)$, recording the last received symbol for the next iteration.

After updating the strings $w_{i,j}$ and $s_{i,j}$ according to the above three scenarios, the p_i computes the next tree-code symbol $\mathcal{T}(s_{i,j})[r]$ and communicates it to p_j (lines 14, 20 and 26). The exchange of all these symbols is performed simultaneously via the A2A scheme.

3.4 Consistency Check, Parse, and Index Generation

It remains to describe how a party checks whether consistent and simultaneous simulation has occurred so far. To check simultaneous simulation, p_i needs to keep track of the round of the original protocol π that it simulated in any given iteration of the simulation protocol. This is done through the `GenIndex` procedure detailed in Algorithm 4. The `GenIndex` procedure takes any of the strings $s_{i,j}, \hat{w}_{i,j}$ as input, and returns, for each element in $s_{i,j}$ or $\hat{w}_{i,j}$, the round of π that corresponds to that element in the simulation. The procedure works as follows. The output will be given via a string of round numbers `Index`, initialized to the all zeros string, and updated serially starting from the first location. At index location l , if $s_{i,j}[l] \in \{0, 1\}$, then the algorithm assigns the value `Index` $[l-1] + 1$

to $\text{Index}[l]$ (assuming $\text{Index}[0] = 0$). If $s_{i,j}[l] = \text{H}$, then $\text{Index}[l]$ is assigned the value $\text{Index}[l - 1]$. This indicates that in the l -th iteration, the party p_i has decided to halt the simulation and wait for other parties to rewind back to the current simulated round. Finally, if $s_{i,j}[l] = \text{BK}$, then $\text{Index}[l]$ is assigned with the value $\text{Index}[l] - 1$, indicating that the last simulated round has been deleted. Note that the output of $\text{GenIndex}(s_{i,j})$ is independent of the choice of the neighbour p_j as Algorithm 3 simultaneously assigns H or BK to all strings $s_{i,j}$ in a given simulation round. On the other hand $\text{GenIndex}(\hat{w}_{i,j})$ may differ between different p_j s.

Algorithm 4 The GenIndex procedure

input: $s \in S^l$

1: **initialize:** $\text{Index} \leftarrow 0^l, i \leftarrow 0$

2: **for** $j = 1$ to l **do**

3: **if** $s[j] \in \{0, 1\}$ **then**

4: $\text{Index}[j] \leftarrow i + 1$

5: $i \leftarrow i + 1$

6: **else if** $s[j] = \text{H}$ **then**

7: $\text{Index}[j] \leftarrow i$

8: **else** ($s[j] = \text{BK}$)

9: $\text{Index}[j] \leftarrow \max\{i - 1, 0\}$ ▷ Do not rewind below 0

10: $i \leftarrow \text{Index}[j]$

11: **end if**

12: **end for**

13: **return** $\text{Index}[1, l]$

Next, p_i deduces how many rounds of π have been simultaneously simulated so far, and the simulation iteration they have been generated in. To do so, it executes the $\text{PARSE}()$ procedure, which is detailed in Algorithm 5. The PARSE procedure takes as inputs the strings $s_{i,j}, w_{i,j}, \hat{w}_{i,j}$ for every p_j , and generates respectively three sets of parsed strings $\sigma_{i,j}, z_{i,j}, \hat{z}_{i,j}$. The parsed strings include only undeleted simulated rounds (i.e., they do not include the control directives H and BK, nor rounds that were rewound due to a BK command), and for each such simulated rounds, they also include the iteration in which that round was simulated, according to the GenIndex output.

The PARSE procedure behaves as follows. A counter for the last simultaneously simulated round, denoted by minInd is initialized to 0. Now, for every round $l \leq r - 1$, the algorithm checks whether indices $\text{Ind}_i[l]$ and $\text{Ind}_{i,j}[l]$ all equal $\text{minInd} + 1$, and whether all the strings $s_{i,j}$ and $\hat{w}_{i,j}$ hold information symbols at location l . If so, this means that a new round was simultaneously simulated at round l . Accordingly, $z_{i,j}, \sigma_{i,j}, \hat{z}_{i,j}$ are appended respectively with $(w_{i,j}[l], l), (s_{i,j}[l], l)$, and $(\hat{w}_{i,j}[l], l)$; the first item in each tuple is the simulated information bit and the second item is the iteration in which this bit was simulated. Also, minInd is increased by 1. Otherwise, if it turns out that $\text{Ind}_i[l] < \text{minInd}$, this implies that some rounds which were previously thought to have been simultaneously simulated were deleted. Accordingly, the last $\text{minInd} - \text{Ind}_i[l]$ symbols from each of the parsed strings $z_{i,j}, \sigma_{i,j}, \hat{z}_{i,j}$, for all $j \neq i$ are deleted. Also, minInd is set to $\text{Ind}_i[l]$, which the party i believes to be last simultaneously simulated rounds. After its completion, the algorithm returns the parsed strings $z_{i,j}, \sigma_{i,j}, \hat{z}_{i,j}$, for all $j \neq i$.

Algorithm 5 The PARSE procedure (for p_i)

input: For all $j \neq i$: $s_{i,j} \in S^{r-1}$, $\hat{w}_{i,j} \in S^{r-1}$, $w_{i,j} \in S^{r-2}$ (for some $r \geq 1$) $\triangleright S^k := \{\emptyset\}$ for $k \leq 0$

initialize: $\hat{z}_{i,j}, z_{i,j}, \sigma_{i,j} \leftarrow \emptyset$, $\text{minInd} \leftarrow 0$, $l \leftarrow 1$

1: $\text{Ind}_i \leftarrow \text{GenIndex}(s_{i,j})$ for some $j \neq i$ $\triangleright$ The choice of j does not affect the output
2: $\text{Ind}_{i,j} \leftarrow \text{GenIndex}(\hat{w}_{i,j}), \forall j \neq i$

3: **while** $l \leq r - 1$ **do**
4: **if** for all $j \neq i$: $\text{Ind}_i[l] = \text{Ind}_{i,j}[l]$ **then**
5: **if** $\text{Ind}_i(l) = \text{minInd} + 1$ **and** for all $j \neq i$: $\hat{w}_{i,j}[l], s_{i,j}[l] \in \{0, 1\}$ **then**
6: $\hat{z}_{i,j} \leftarrow \hat{z}_{i,j} \circ (\hat{w}_{i,j}[l], l)$
7: $\sigma_{i,j} \leftarrow \sigma_{i,j} \circ (s_{i,j}[l], l)$
8: **if** $l \geq 2$ **then** $z_{i,j} \leftarrow z_{i,j} \circ (w_{i,j}[l-1], l-1)$ **end if**
9: $\text{minInd} \leftarrow \text{minInd} + 1$
10: **else if** $\text{Ind}_i[l] < \text{minInd}$ **then**
11: Truncate the last $(\text{minInd} - \text{Ind}_i[l])$ symbols of $z_{i,j}, \hat{z}_{i,j}, \sigma_{i,j}$, for all $j \neq i$.
12: $\text{minInd} \leftarrow \text{Ind}_i[l]$
13: **end if**
14: **end if**
15: $l \leftarrow l + 1$
16: **end while**

17: **return** $\sigma_{i,j}, \hat{z}_{i,j}, z_{i,j}$, for all $j \neq i$

To complete the description of the consistency check, we first introduce some notation. Recall that the l -th index of a parsed string (say, $\sigma_{i,j}$) contains a tuple (a, b) , where the first element is an information bit and the second element is an iteration number. We denote these two, respectively, by $\text{Info}(\sigma_{i,j}[l])$ and $\text{Rnd}(\sigma_{i,j}[l])$.

After obtaining the parsed strings (line 4 in Algorithm 3), p_i checks whether they are all consistent with protocol π , and whether simultaneous simulation has occurred so far. This is performed by the `ConsCheck()` procedure depicted in Algorithm 6. This procedure simply checks whether the parsed string of all symbols received so far, i.e., $\hat{z}_{i,j}$, matches with the parsed recorded received transcript $z_{i,j}$ (used to generate previous sent symbols), for all p_j s. We note that unlike the original scheme in [RS94], we do not have to check whether $|\hat{z}_{i,j}| > |z_{i,j}|$, since the strings are parsed simultaneously by the PARSE procedure, which ensures that $|\hat{z}_{i,j}| = |z_{i,j}| + 1$, with the one extra symbol being the latest received information symbol. Furthermore, we have $\text{Info}(\sigma_{i,j}[l]) = \pi(x_i, \{\text{Info}(z_{i,j}[1, l-1])\}_{j \neq i})$ by default, directly from Algorithm 3.

If the consistency check fails, p_i sends BK to all its neighbors in this iteration (lines 22 to 27). If, however, the test passes, then p_i checks for simultaneous simulation. To do so, it again executes `GenIndex` and obtains Ind_i and $\text{Ind}_{i,j}$ (lines 7 and 8; these were already computed in PARSE). Now, p_i checks if all parties are set to simulate the same round of π , namely, whether $\text{Ind}_i[r-1] = \text{Ind}_{i,j}[r-1] = |\sigma_{i,j}|$ for all $j \neq i$. In this case, p_i simulates the next round of π (line 9), and otherwise, it either holds (line 16) or rewinds (line 22). When $\text{Ind}_i[r-1] = |\sigma_{i,j}|$ with all neighbors p_j not being behind, $\text{Ind}_{i,j}[r-1] \geq |\sigma_{i,j}|$, and at least one p_j being ahead satisfying $\text{Ind}_{i,j}[r-1] > |\sigma_{i,j}|$, party p_i holds at its current position and sends H to all its neighbours (who will in turn rewind to the longest shared simultaneous prefix). If none of these hold (i.e., at least one p_j is behind) p_i rewinds towards that round by sending BK to all its neighbours, indicating that it is deleting the last simulated step.

Algorithm 6 The ConsCheck procedure (for p_i)

inputs: For all $j \neq i$: $\sigma_{i,j}, \hat{z}_{i,j}, z_{i,j}$

initialize: flag $\leftarrow 1$

```
1:  $l \leftarrow |\sigma_{i,j}|$  for some  $j \neq i$  ▷ The choice of  $j$  does not affect the output
2: for  $q = 1$  to  $l - 1$  do
3:   if  $\exists j \neq i$  such that  $z_{i,j}[q] \neq \hat{z}_{i,j}[q]$  then
4:     flag  $\leftarrow 0$  ▷ inconsistency found
5:   end if
6: end for

7: return flag
```

3.5 Dummy Phase and Output

After running $2R$ simulation iterations (line 2 of Algorithm 3), the parties continue with additional R dummy iterations (line 30). In these iterations, the parties only send tree code symbols extending their sent transcript by zeros. The reason for this dummy phase is to avoid errors in decoding the suffix of the tree code. Indeed, we prove in appendix C.2 that with high probability, after $2R$ iteration of the simulation, the transcript encoded by the tree code is correct. However, channel noise that occur around iteration $2R$ could prevent parties from decoding the complete string, so that the suffix of the string is corrupted (see suffix distance properties of tree codes in [Gel17]). The dummy phase guarantees that even if a suffix of length $R - 1$ is corrupted (which can happen with negligible probability), the $2R$ -long prefix can still be correctly recovered. While one could potentially just run the loop of line 2 for R additional iterations, adding this dummy phase greatly simplifies the analysis (See appendix C.2).

After the dummy phase is over, p_i decodes the tree coded strings sent by all its neighbors (line 33) and obtains $\hat{w}_{i,j} \in S^{3R}$. Next it parses the $2R$ first symbols (all the rest are 0s and can be ignored) of $s_{i,j}, w_{i,j}, \hat{w}_{i,j}$, for all $j \neq i$ (line 34). and obtains the parsed sent and received transcripts, respectively, $\sigma_{i,j}, z_{i,j}$. Finally, p_i truncates each of these transcripts to length $R = |\pi|$ and outputs them all (line 35).

Formal analysis of this IC scheme is presented in Appendix C.

4 Combining the Chunked Simulation with Simultaneous Simulation

It is straightforward to verify that by combining the ideas of the chunked scheme in Section 3.1 with the simultaneous scheme of Section 3.2, one can obtain a simultaneous simulation scheme that simulates chunks of size c bits at a time rather than a single bit. To avoid excessive repetition, we provide an overview of the merged IC scheme and the related proof technique in Appendix D, and only state here the resulting theorem.

Theorem 7. *For n parties connected by pairwise BSC_ε links with $\varepsilon < 1/2$, for any constant parameter $c \in \mathbb{N}$, and for any noiseless protocol π , there exists a simulation protocol $\tilde{\pi}$ which ensures correct and simultaneous simulation of π in $O_\varepsilon(|\pi|)$ rounds. The simulation succeeds with probability $1 - 2^{-\Omega(|\pi|\sqrt{n})}$, over the randomness of the channel. Furthermore, the scheme $\tilde{\pi}$ works in chunks of size c bits, namely, each “chunk” is simultaneously simulated by all parties.*

5 Multi-Party Computation over BSC Channels

In the general multi-party computation (MPC) functionality, the parties compute a public boolean circuit $\mathcal{C}$ on private inputs $x_1, \dots, x_n$, such that each party learns (only) the output $\mathcal{C}(x_1, \dots, x_n)$. In Section 6, we describe how to slightly modify our construction to ensure that only specific parties learn specific output bits. We first outline the original GMW protocol for MPC over noiseless channels, followed by our proposal for a new “GMW-like” protocol for MPC that is secure and has a constant rate over BSC channels.

The original GMW construction. As a starting point, we consider the n -party GMW protocol [GMW87, Gol04] for MPC, assuming the parties are connected via noiseless channels. At a high level, the protocol proceeds as follows:

- **Input Sharing Phase:** In this phase, any party P_i with input x_i creates a (n, n) -additive sharing of their inputs: they pick random bits x_{ij} such that $x_i = \sum_{j=1}^n x_{ij}$. Party P_i then sends each share x_{ij} to party P_j , ensuring that each input is secretly shared between the n parties.
- **Circuit Evaluation Phase:** For each NAND gate in circuit f in topological order, the parties evaluate each gate in the circuit, generating secret shares of the output wire given the shares of the input wires. The protocol evaluates this securely using two OT calls between every pair of parties.
- **Output Reconstruction Phase:** When all NAND gates are processed, the parties share their additive share with every other party for each output wire. Adding these shares for each output wire gives the corresponding output.

This original GMW protocol is secure in the $\mathcal{F}_{\text{ROT}}$ -hybrid model against passive adversaries corrupting at most $\leq n - 1$ parties. Additionally, the GMW circuit evaluation phase can be viewed as a protocol with correlation (Definition 4), where the correlation index = 6 and the set of correlations include two instances of pairwise ROT correlations, as required in a single instance of NAND evaluation. Consequently, the GMW protocol remains correct regardless of which specific instantiations of ROT correlations are used for each step in the evaluation phase.

Proposed protocol outline. To construct our noise-resilient MPC protocol, we use a standard ECC scheme and a new IC scheme proposed in Theorem 7. At a high level, our proposed construction proceeds as follows:

- **Input Sharing Phase:** Each party creates (n, n) -additive secret shares of each input, similar to the GMW protocol. Each share is sent over the BSC channel using an ECC with length $\Theta_\epsilon(\kappa)$. The probability of failure of each ECC is $2^{-\Theta_\epsilon(\kappa)}$. Hence, with all but negligible probability in κ , the decoded received shares by each party will match exactly the shares from the input sharing phase of the original GMW protocol over noiseless channels. The communication complexity of this step is $O_{n,\epsilon}(m + \kappa)$, where m is the number of input bits per party.
- **Circuit Evaluation Phase:** Use the proposed coding scheme (Theorem 7) to execute only the circuit evaluation phase of GMW, which can be viewed as a protocol with correlated randomness from ROT. The coding scheme has a constant rate. Hence, the communication complexity of this step is $O_{n,\epsilon}(|\mathcal{C}| + \kappa)$. By the correctness of the underlying IC scheme, the output of the evaluation phase will be correct with probability $1 - 2^{-\Theta(|\mathcal{C}| + \kappa)}$. Hence, the parties end up with the correct xor shares for each output wire in the circuit $\mathcal{C}$.

- **Output Reconstruction Phase:** Similar to the input sharing phase, each share is exchanged using a standard ECC.

A formal description of this protocol ($\Pi_{\text{MPC}}^{\text{BSC}}$) is provided in Figure 5. As noted in the above discussion, it is easy to check that the protocol is correct and circuit-scalable. However, the privacy of the protocol is not as apparent. As a first step towards the privacy proof, we analyze just a single NAND execution during the IC scheme simulation and show that it satisfies a notion of privacy even when the protocol is not always correct. We use this building block to construct our simulator for protocol $\Pi_{\text{MPC}}^{\text{BSC}}$.

5.1 NAND over BSC

In the proposed IC scheme with chunk size c , in each iteration, the parties run c rounds of the protocol sequentially, each using the A2A communication pattern. Hence, in each iteration/chunk of our protocol $\Pi_{\text{MPC}}^{\text{BSC}}$, the parties execute the protocol $\text{A2A}(\Pi_{\text{NAND}})$, where Π_{NAND} corresponds to the GMW NAND protocol and the A2A compiler sends each message of the input protocols through a sequence of A2A calls. In this section, we first outline the GMW NAND protocol, followed by an analysis of the privacy of $\text{A2A}(\Pi_{\text{NAND}})$ protocol over BSC channels.

NAND execution in GMW. We define a Π_{OT} protocol (presented in Figure 2) that implements the standard OT in the $\mathcal{F}_{\text{ROT}}$ hybrid model. Note that this primitive is not secure over noisy channels. In particular, due to the BSC noise, the receiver might learn the bit it didn't select, i.e., x_{1-b} . However, we use this to construct our multi-party NAND protocol in Figure 3.

Protocol Π_{OT}

Inputs: Party P_j 's input is a bit b ; Party P_i 's input is a pair of bits (x_0, x_1) .

1. Parties make a call to $\mathcal{F}_{\text{ROT}}$: Party i gets x'_0, x'_1 , while Party P_j gets b' and $x'_{b'}$ as output.
2. Party P_j sends $c = b + b'$; Assume Party i receives c' .
3. Party P_i sends $x_0 + x'_{c'}$. Assume Party j receives y_0 .
4. Party P_i sends $x_1 + x'_{1-c'}$. Assume Party j receives y_1 .
5. P_j outputs $y_b + x'_{b'}$ as her output.

Fig. 2: The Π_{OT} Protocol

In this NAND protocol each party P_i has input (x_i, y_i) for $i \in [n]$ where $\{x_i\}_i$ is a sharing of some x and $\{y_i\}_i$ is a sharing of some y . Then the NAND of bits x and y is defined as:

$$1 + (x_1 + x_2 + \dots + x_n)(y_1 + y_2 + \dots + y_n) = 1 + \left(\sum_i x_i y_i \right) + \left(\sum_i \sum_{j \neq i} (x_i y_j) \right).$$

Party i can compute $x_i y_i$ locally. To compute shares of $x_i y_j + x_j y_i$ for $i < j$, parties P_i and P_j do

- Party i , playing the role of sender, with inputs $(r_1^{ij}, y_i + r_1^{ij})$ to the OT functionality, where r_1^{ij} is chosen uniformly at random. Party j , playing the receiver role, inputs choice bit x_j to the OT functionality and receives o_1^{ij} .

- Party i , playing the role of sender, sends $(r_2^{ij}, x_i + r_2^{ij})$ to the OT functionality, where r_2^{ij} is chosen uniformly at random. Party j , playing the role of receiver, sends y_j to the OT functionality and receives o_2^{ij} . Note that $r_1^{ij} + r_2^{ij} + o_1^{ij} + o_2^{ij} = x_i y_j + x_j y_i$.

For all i , let z_{ij} denote the value of $r_1^{ij} + r_2^{ij}$ for all $j > i$. Let z_{ij} denote the value of $o_1^{ij} + o_2^{ij}$ for all $j < i$. Now, each party i computes $z_i = x_i \cdot y_i + \left(\sum_{j \neq i} z_{ij}\right)$ for $i > 1$ and $z_1 = x_1 \cdot 1 + y_1 + \left(\sum_{j \neq 1} z_{ij}\right)$. These values form shares of $x \cdot y$.

Protocol Π_{NAND}

- **Inputs:** For $i \in [n]$, Party P_i inputs $x_i \in \{0, 1\}$ and $y_i \in \{0, 1\}$
- **NAND Gate Protocol**
 1. Each pair of parties execute $n(n-1)/2$ instances of Π_{OT} (see Figure 2) in parallel, twice, in the following manner. Party i does the following.
 - Party i , playing the role of sender, sets $(r_1^{ij}, y_i + r_1^{ij})$ where r_1^{ij} is a random bit, as its input to Π_{OT} , for each party $j > i$. Party i , playing the role of receiver, sets x_i as its input to Π_{OT} , for each party $j < i$.
 - Let the output of party i playing as the receiver with party j in Π_{OT} be o_1^{ij} for $j < i$.
 - Party i , playing the role of sender, sets $(r_2^{ij}, x_i + r_2^{ij})$ where r_2^{ij} is a random bit, as its input to Π_{OT} , for each party $j > i$. Party i , playing the role of receiver, sets y_i as its input to Π_{OT} , for each party $j < i$.
 - Let the output of party i playing as the receiver with party j in Π_{OT} be o_2^{ij} for $j < i$.
 2. Party i sets $z_{ij} = r_1^{ij} + r_2^{ij}$ for $j > i$ and sets $z_{ij} = o_1^{ij} + o_2^{ij}$ for $j < i$.
- **Output Computation.** Each party i computes $z_i \leftarrow x_i y_i + \sum_{j \neq i} z_{ij} + 1[i \stackrel{?}{=} 1]$. These values form shares of $1 + x \cdot y$.

Fig. 3: Protocol Π_{NAND}

Theorem 8. *For any set of corrupt parties $C \subset [n]$, the n -party NAND protocol $\Pi' = \text{A2A}(\Pi_{\text{NAND}})$ over BSC channel satisfies the transparent-channel privacy property: there exist a simulator $\text{Sim}_{\text{NAND}}^{\text{BSC}}$ such that for any inputs $\{x_i, y_i \in \mathbb{Z}_2\}_{i \in [n]}$ the following holds:*

$$\text{Sim}_{\text{NAND}}^{\text{BSC}}(C; \{x_i, y_i\}_{i \in C}) \equiv \text{View}_{\Pi'}^{\text{BSC}}(\{x_i, y_i\}_{i \in [n]})$$

Proof. The simulator description is presented in Figure 4. The Simulator essentially simulates the A2A NAND protocol $\text{A2A}(\Pi_{\text{NAND}})$ for each corrupt party, where each message sent by an honest party is a freshly sampled random bit.

The augmented view of the C corrupt parties in the real protocol execution is defined as

$$\text{View}'_{\Pi'}(C; x, y) = \{x_C, r_C, \text{corr}_C, M_1, M_2, \dots, M_n\}$$

where M_i are the messages received by party i during the execution of the protocol, and corr correspond to 2 instances of pairwise random OT correlations.

We show the simulator is perfectly indistinguishable from the view of the A2A NAND protocol by showing the input to the A2A protocol has the same distribution in both the protocol and the simulator.

For any $i, j \in [n]$ with $i < j$ parties P_i and P_j execute two instances of OT in the NAND protocol, where the first instance has the following form: P_i acts as the receiver with bit x_i and it has $(u_{b'}, b')$ OT correlation. Party P_j acts as the sender with messages $(m_0, m_1) = (r_1^{ij}, y_i + r_1^{ij})$ with OT correlation (u_0, u_1) ,

- If $i, j \in C$: the simulator $\text{Sim}_{\text{NAND}}^{\text{BSC}}$ picks the same messages as input to A2A as the protocol.
- If $i \in C, j \notin C$: P_i 's input messages to A2A can be exactly simulated. In the protocol, P_j sends $(m_0 + u_{c'}, m_1 + u_{1-c'})$ to P_i via A2A. This can be simulated by a pair of random independent bits, where the randomness is over u_{1-b} and r_i^{ij} - random bits not in the view of P_j .
- If $i \notin C, j \in C$: P_j 's input messages to A2A can be exactly simulated. P_i 's only input message to A2A in the protocol for P_j is the bit $(b + b')$. This can be simulated by a random bit since b' is a random bit independent of all other messages sent in the protocol.
- $i, j \notin C$: With the same argument as the above two cases, the input messages of parties P_i and P_j can be simulated by random bits.

The same argument also shows that the view of the second OT execution via A2A has the same distribution in both the protocol and the simulator. □

Simulator $\text{Sim}_{\text{NAND}}^{\text{BSC}}(C, x_C, y_C)$

1. For $p \in \{0, \dots, 6\}$: (for each round of NAND)
 - Simulate A2A protocol over BSC_ε channels where each corrupt party picks their A2A input as specified by p^{th} round in the $\text{A2A}(\Pi_{\text{NAND}})$ protocol and each honest party picks a random bit
2. Output simulated augmented corrupted party views

Fig. 4: Simulator Sim_{NAND}

The proposed simulator is a randomized algorithm that tosses random coins to sample noises in the BSC channels and tosses random coins to simulate each party's internal randomness. In the proposed MPC simulator in Figure 6, we would need to explicitly input the randomness due to BSC channel noises. Hence, we use the extended notation $\text{Sim}_{\text{NAND}}^{\text{BSC}}(C; \{x_i, y_i\}_{i \in C}; r)$ for the NAND simulator, where the added parameter r is a bit string with size matching the total communication bits in the protocol, and $r_i \leftarrow \mathcal{N}(\varepsilon)$ for all i .

When r is an all-zero string, it represents executing the A2A-NAND protocol over non-noisy channels. In this case, the underlying NAND protocol is correct and private, and its proof follows directly from the NAND security in GMW [GMW87, Gol04]. This also directly implies the security of A2A-NAND over non-noisy channels as well:

Theorem 9. *For any set of corrupt parties $C \subset [n]$, the n -party NAND protocol $\Pi' = \text{A2A}(\Pi_{\text{NAND}})$ is channel transparent and it securely implements the NAND functionality over non-noisy channels, i.e. for any inputs $\{x_i, y_i \in \mathbb{Z}_2\}_{i \in [n]}$ the following holds:*

$$(\text{Sim}_{\text{NAND}}(C; \{x_i, y_i, z_i\}_{i \in C}), \{z_i\}_{i \in [n]}) \equiv (\text{View}_{\Pi'}(\{x_i, y_i\}_{i \in [n]}), \text{Output}_{\Pi'}(\{x_i, y_i\}_{i \in [n]}))$$

where

$$\text{For } i \in [n] : z_i \leftarrow_{\$} \{0, 1\} \text{ conditioned on } \sum_{i=1}^n z_i = 1 + \left(\sum_{i=1}^n x_i\right) \left(\sum_{i=1}^n y_i\right).$$

5.2 Security Proof for $\Pi_{\text{MPC}}^{\text{BSC}}$

The simulator maintains a variable rcvMsg_i for each $i \in [n]$, and it contains the simulated received messages by party P_i in the protocol.

In the proposed IC scheme (Theorem 3), in each iteration, all parties simulate some contiguous c rounds of the protocol via A2A, followed by an exchange of tree code symbols via A2A. Based on the received protocol messages and the tree code symbol, each party either redos the computation for the current chunk of rounds, goes to the next chunk, or backtracks to the previous chunk of rounds in the next iteration. Hence, in protocol $\Pi_{\text{MPC}}^{\text{BSC}}$, the protocol executes a single NAND protocol in each chunk, followed by an exchange of tree code symbols via A2A. To simplify the presentation in the simulator, we present three deterministic functions corresponding to the local computation performed by each party during any chunk/iteration:

- $g_i \leftarrow \text{nextNANDgate}(t, \text{rcvMsg}_i)$: takes as input the iteration index t and all the messages received by the party P_i . It outputs the NAND gate index that it executes in the t^{th} iteration. Note that this is a function of just the received messages and is independent of P_i 's input.
- $(a_i, b_i) \leftarrow \text{nextNANDinp}(t, x_i, r_j, \text{corr}_i \text{rcvMsg}_i)$: this function outputs the NAND inputs of party P_i in iteration t given the party's input x_i , its correlated randomness used so far corr_i , its internal randomness r_i and the messages received so far rcvMsg_i .
- $t \leftarrow \text{nextSymbol}(t, \text{rcvMsg})$: This outputs the tree code symbol that the party P_i sends in iteration t as a function of its set of received messages so far.

Of the above three algorithms, note that only the nextNANDinp directly depends on the party's inputs. The other two functions nextNANDgate and nextSymbol are a function of just the received messages.

We also define a helper function $\text{SameOutGate}()$, which takes as input a set of n gates and a circuit $\mathcal{C}$ and outputs a boolean value **true** if all input gates match and are an output gate in $\mathcal{C}$, else the function returns **false**.

Figure 6 presents the formal simulator for protocol $\Pi_{\text{MPC}}^{\text{BSC}}$. At a high level, the simulator proceeds as follows:

- **Initialization phase**: Here, the simulator essentially follows the same protocol as the initialization phase of $\Pi_{\text{MPC}}^{\text{BSC}}$. Any subset of less than or equal to $n - 1$ shares of a (n, n) -additive secret sharing is identically distributed for any two secrets. Hence, the views generated by the simulator here are indistinguishable from the protocol.
- **Evaluation phase**: ‘The simulator executes $\lfloor \pi_f \rfloor / 3$ iterations/chunks, where in each iteration it simulates a single A2A-NAND execution, where π_f is the GMW evaluation protocol. Let T represent the total number of bits of communication in the protocol $\text{A2A}(\Pi_{\text{NAND}})$. Then, the simulator first samples T bits of noise (each an element of $\text{bernoulli}(\varepsilon)$). When this sampled noise is not all zeroes and not all parties are simulating the same NAND gate (as decided by nextNANDgate), the simulated view is generated using $\text{Sim}_{\text{NAND}}^{\text{BSC}}$ given corrupt party inputs. If all parties execute the same output gate (say g) and no BSC noise is added, then we simulate the protocol views using Sim_{NAND} given corrupt party inputs and ideal output of gate g . The tree code symbols exchanges via A2A are a deterministic function of only the received messages. Hence, the simulator can also generate this view, given that the received messages are simulated for all the parties.
- **Reconstruction phase**: For each output gate, parties send ECC encoding of their output gate shares as the coding scheme dictates.

Protocol Π_{MPC}

- **Inputs:** A (public) circuit $f = \mathcal{C}$ and inputs $x_1, x_2, \dots, x_n$ held by party $1, 2, \dots, n$, respectively.
- **Initialization:**
 - For every party $1 \leq i \leq n$, i sends its encoded share of its inputs x_{ij} using an ECC (guaranteed by Lemma 2) of length $O(\kappa)$ with decoding error $\exp(-\kappa)$, for $1 \leq j \leq m$. It also receives and decodes the (encoded) shares of other parties.
- **Evaluation:**
 - Let π_f be the GMW protocol that describes the evaluation of the NAND gates where the parties evaluate each of the NAND gates in a predetermined topological order.
 - Apply the interactive coding scheme of Theorem 7 with chunk size $c = 6$ for the evaluation of π_f with correlated randomness ROT. The chunk size 6 ensures precisely one NAND gate is simulated in each chunk of the protocol execution.
- **Reconstruction:**
 - If the interactive coding does not succeed, all parties output $\perp$.
 - Otherwise, for each output gate g , all parties send their shares to each other using ECC of length $O(\kappa)$.
 - Each party obtains the expected output by adding its share with the received shares for each output gate

Fig. 5: Protocol Π_{MPC}

Simulator $\text{Sim}_{\text{MPC}}^{\text{BSC}}(C; x_C; y = \mathcal{C}(x))$

Initialization

1. **For** each $i \in [n]$:
 - Initialize $\text{rcvMsg}_i = \phi, t_i$
 - Sample random bits $x_{i,j} \leftarrow_{\$} \{0, 1\}$ for $j \in [n-1]$
 - **if** $i \in C$: Set $x_{i,n-1}$ such that $x_i = \sum_{j \in [n]} x_{i,j}$
 - **Else**: Set $x_{i,n-1} \leftarrow_{\$} \{0, 1\}$
2. For all $i, j \in [n]$, $i \neq j$: append to rcvMsg_i : $\text{ECC}(x_{ij}) + \mathcal{N}$ as the received messages from party j , where ECC is an error-correcting code with length $O(\kappa)$ (see Lemma 2) and $\mathcal{N} \leftarrow \text{bernoulli}(\varepsilon)$.

Evaluation

1. Simulate a single chunk of $c = 6$ rounds i.e a single Π_{NAND} execution: For each party i , do the following for $l = 1$ to $\frac{2|\pi_f|}{c}$:
 - (a) For NAND execution in the current chunk:
 - (b) For $i \in [n]$: $g_i \leftarrow \text{nextNANDgate}(\text{rcvMsg}_i)$
 - i. For each corrupt part $j \in C$: $(a_j, b_j) \leftarrow \text{nextNANDinp}(x_j, r_j, \text{rcvMsg}_j)$
 - ii. Initialize T -bit string r : for each i : $r_i \leftarrow \mathcal{N}(\varepsilon)$
 - iii. If $\text{SameOutGate}(\mathcal{C}, \{g_i\}_{i \in [n]}) \vee (r \neq 0^T)$:
 - Update rcvMsg_i with the output of $\text{Sim}_{\text{NAND}}^{\text{BSC}}(C; \{a_i, b_i\}_C)$
 - iv. Else:
 - Let y_g be the ideal output corresponding to gate g
 - For $i \in [n]$, sample $o_i^g \leftarrow_{\$} \{0, 1\}$ conditioned on $\sum_{i=1}^n o_i^g = y_g$

- Update View_i with the output of $\text{Sim}_{\text{NAND}}(C; \{a_i, b_i\}_C; \{o_i^g\}_{i \in C})$
- v. $t_i \leftarrow \text{nextSymbol}(\text{rcvMsg}_i)$
- vi. Simulate $\log |I|$ calls to A2A protocol where party P_i sends to party P_j bits in the tree code symbol t_{ij} for each $i \neq j$
- vii. For each i , update rcvMsg_i with the views for party i in the above $\log |I|$ calls to A2A protocol

Output Calculation

1. If too many errors occur and the simulated transcript's length is less than $\lceil \pi \rceil$ rounds, output $\perp$.
2. For each output gate g : Each party i sends $\text{ECC}(o_i^g) + \mathcal{N}$ where the noise $\mathcal{N} \leftarrow \text{bernoulli}(\varepsilon)$.

Fig. 6: Simulator $\text{Sim}_{\text{MPC}}^{\text{BSC}}$

Let $\widetilde{\text{View}}_{\text{MPC}}^{\text{BSC}}(C; \kappa, x, t)$ represent the view of the corrupted parties C in the protocol Π_{MPC} with inputs x after executing exactly t iterations/chunks of the evaluation phase. Likewise, let $\widetilde{\text{Sim}}_{\text{MPC}}^{\text{BSC}}(C; \kappa, x_C, f(x), t)$ represent the view generated by the simulator $\text{Sim}_{\text{MPC}}^{\text{BSC}}$ until t iterations of the evaluation phase. For the final security proof, we first prove the following lemma with its proof in the Appendix E.

Lemma 1. *For every $x \in \{0, 1\}^n$, security parameter κ , $I \subset [n]$ and $t \in \llbracket \pi \rrbracket / 3$:*

$$\widetilde{\text{View}}_{\text{MPC}}^{\text{BSC}}(C; x, t) \approx_{\exp(-\kappa)} \widetilde{\text{Sim}}_{\text{MPC}}^{\text{BSC}}(C; x_C, f(x), t)$$

Theorem 10 (Main theorem, Formal). *Fix a security parameter $\kappa > 0$ and a noise parameter $\varepsilon \in (0, \frac{1}{2})$. For any circuit $\mathcal{C} : \{\{0, 1\}^m\}^n \rightarrow \{0, 1\}^{m'}$ (that's poly-sized in κ), there exists a n -party protocol Π_{MPC} (presented in Protocol 5) that evaluates the circuit $\mathcal{C}(x_1, \dots, x_n)$ in the $\mathcal{F}_{\text{ROT}}$ hybrid model over BSC_ε channels that is statistically secure against semi-honest adversaries. Moreover, the communication complexity of Π_{MPC} is $O_{n, \varepsilon}(|C| + \kappa)$.*

Proof. Since we consider deterministic functionality, we can analyze the correctness and privacy separately.

- **Correctness:** By the correctness of the IC scheme (Theorem 7), the evaluation phase of protocol $\Pi_{\text{MPC}}^{\text{BSC}}$ with ROT correlated randomness exactly matches the output of the GMW protocol's evaluation phase for some correlated randomness. Protocol $\Pi_{\text{MPC}}^{\text{BSC}}$ also simulates the original GMW protocol in the initialization and reconstruction phases, where each message is encoded using ECC, which is incorrectly decoded with probability $\text{negl}(\kappa)$. Hence, the correctness of $\Pi_{\text{MPC}}^{\text{BSC}}$ reduces to the correctness of the original GMW protocol.
- **Privacy:** By Lemma 1, the views of the corrupted parties in the protocol and the simulator are indistinguishable until the end of the evaluation phase with $1 - \text{negl}(\kappa)$ probability. In the reconstruction phase, the view of all parties in a real execution of the protocol consists of all the n shares of each output gate (say g), encoded with a ECC of size $O(|C| + \kappa)$. The IC scheme ensures that the final execution of the output NAND gate g was carried out in the same iteration (say t_g) by all parties. By correctness of the IC scheme, no noise is added in iteration t_g . In $\text{Sim}_{\text{MPC}}^{\text{BSC}}$ we use the simulator Sim_{NAND} to generate the A2A-NAND views for precisely the same iterations - when all parties execute the same output gate and no BSC noise is added. Hence,

the n shares for each output gate in the simulator and the protocol corresponding to the output of the NAND gate execution from the same iteration, making the joint view of the reconstruction phase and evaluation phase indistinguishable in both the simulator and the protocol.

- **Communication complexity:** The initialization phase and reconstruction phase have communication costs $O_{\varepsilon,n}(m + \kappa)$ and $O_{\varepsilon,n}(m' + \kappa)$ respectively. Since the underlying IC scheme is a constant rate, the evaluation phase has communication cost $O_{\varepsilon,n}(|C| + \kappa)$.

□

6 Extensions and Further Optimizations

We now discuss several extensions to our results and compare them with other approaches.

MPC with private outputs. We can naturally extend the MPC construction in Theorem 10 with public outputs so that only specific parties get specific output bits. Let $\mathcal{C}(x_1, \dots, x_n) = (y_1, \dots, y_n)$ be a circuit with inputs $x_i \in \{0, 1\}^m$ and outputs $y_i \in \{0, 1\}^{m'}$. If we want to design an MPC protocol over BSC channels such that only P_i learns y_i , we can use the construction in Theorem 10 where each party P_i inputs (x_i, r_i) , where r_i is a random string of length m' . And we use the MPC protocol to compute the circuit $\mathcal{C}^*((x_1, r_1), \dots, (x_n, r_n)) = ((y_1 + r_1), \dots, (y_n + r_n))$ where $\mathcal{C}(x_1, \dots, x_n) = (y_1, \dots, y_n)$.

Since only party P_i knows random string r_i , only P_i can learn the output y_i given the public output of $\mathcal{C}^*$ from the MPC protocol. Its also easy to verify that this transformation doesn't affect the rate of the MPC protocol.

Reducing the round complexity. For any boolean circuit $\mathcal{C}$, the classic GMW protocol gives a secure MPC protocol over noiseless channels with round complexity $O(\text{depth}(\mathcal{C}))$. Whereas our GMW-inspired MPC protocol over BSC channels has round complexity $O(|\mathcal{C}|)$ - since it sequentially simulates one NAND gate in each iteration of the IC protocol. We present a natural transformation here that can reduce the round complexity of the MPC protocol over noisy channels to $O(\kappa \cdot (\text{depth}(\mathcal{C}) + \kappa))$. For any circuit $\mathcal{C}$, we can decompose it into a sequence of layers, each layer corresponding to all gates at a fixed depth in the circuit. We define a layer to be *shallow* if it has less than κ NAND gates, else we term it *wide*. Now we can decompose $\mathcal{C}$ into disjoint circuits $\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_l$, each of depth $d_1, \dots, d_l$ respectively for some $l \in \mathbb{N}$ such that the following holds:

- $\mathcal{C}_1$ includes all gates in the first d_1 layers of $\mathcal{C}$, $\mathcal{C}_2$ contains all gates in the next d_2 layers of $\mathcal{C}$, and so on.
- $\sum_{i=1}^l d_i = d$
- For any even i , each layer of circuit $\mathcal{C}_i$ contains at least κ gates.
- For any odd i , each layer of circuit $\mathcal{C}_i$ contains less than κ gates.

By definition, such a decomposition is unique for any circuit $\mathcal{C}$, where the first sub-circuit $\mathcal{C}_1$ is allowed to be empty. Hence, this transformation decomposes the original circuit into a sequence of shallow and wide circuits which are odd and even labeled respectively. We note that when evaluating circuit $\mathcal{C}$, each gate in $\mathcal{C}_i$ is evaluated before any gate in $\mathcal{C}_{i+1}$ for any i . Next, we define our modified GMW evaluation protocol over BSC channels for circuit $\mathcal{C}$:

- For each shallow circuit ($\mathcal{C}_i$ for odd i) we can use the proposed multi-party IC scheme (Theorem 7) to evaluate the corresponding GMW evaluation protocol over BSC. This gives a protocol with communication and round complexity $O(|\mathcal{C}_i| + \kappa)$.

- For each wide circuit ($\mathcal{C}_i$ for even i), we execute its GMW evaluation phase by encoding each GMW protocol message with a standard ECC code (of length $O(\kappa)$). Since each protocol message in this wide-circuit GMW evaluation has $O(\kappa)$ bits, this subprotocol is circuit-scalable, and it has $O(\text{depth}(\mathcal{C}_i))$ round complexity.

In short, we use our proposed IC schemes for evaluating shallow circuits and standard ECC for evaluating wide circuits where each message has $O(\kappa)$ bits in the GMW protocol. This entire transformation yields a circuit-scalable MPC protocol over noisy channels with round complexity $O(\sum_{\text{odd } i} (|\mathcal{C}_i| + \kappa) + \sum_{\text{even } i} \text{depth}(\mathcal{C}_i))$. Given the decomposition structure, we have $|\mathcal{C}_i| \leq \kappa \cdot \text{depth}(\mathcal{C}_i)$ for each odd i or shallow circuit $\mathcal{C}_i$. Hence, the round complexity is bounded by

$$\begin{aligned} O\left(\sum_{\text{odd } i} \kappa \cdot (\text{depth}(\mathcal{C}_i) + \kappa) + \sum_{\text{even } i} \text{depth}(\mathcal{C}_i)\right) &= O\left(\kappa \cdot \left(\sum_i \text{depth}(\mathcal{C}_i) + \kappa\right)\right) \\ &= O(\kappa \cdot (\text{depth}(\mathcal{C}) + \kappa)) \end{aligned}$$

Other noisy channels. While our IC scheme is designed for BSC channels, it can be readily extended to other natural noise models, including *elastic* and *erasure* channels, which we briefly define below.

An (α, β) -elastic channel [DFMS04, Wul09] is similar to BSC channel where each bit sent on the channel is flipped with some fixed probability α . However, it introduces an asymmetry: a corrupt receiver can increase the probability with which it can receive the correct bit sent on the channel. This is formally captured by reducing the bit-flip probability to β ($< \alpha$) for a corrupt receiver. Our proposed IC scheme and its analysis over BSC from Theorem 10 naturally extend to elastic networks given as building block a constant-rate batch OT construction over elastic networks, which is known from [KMS16, CDLR16]. For more details, we point the reader to Theorem 21 in [GPZ18], which presents a similar extension from BSC to elastic networks in the context of 2PC.

Another very natural model of noisy networks is an erasure channel, where each transmitted bit is either received correctly or erased (i.e., lost) with some fixed probability. Since erasure channels are strictly weaker than BSC channels, our construction also extends to this stochastic model. In fact, it is plausible that even simpler IC schemes may suffice in this setting. However, extending our approach to (mildly) adversarial noisy channels remains a challenging open problem, which we leave for future work.

7 Conclusions and Future Work

We put forth the study of information-theoretic MPC over noisy networks for more than two parties. We constructed the first MPC protocol that is secure against a semi-honest majority with an optimal communication overhead in circuit size: it computes any circuit $\mathcal{C}$ with communication complexity proportional to its size, i.e., $O_n(|\mathcal{C}|)$. Our protocol departs from the classical MPC methodology by introducing a novel *interactive-coding* that defies channel noise by rewinding the protocol to a round prior to the corrupted transmission. Such rewinding gives rise to two significant challenges: privacy violations and inconsistent correlated randomness. Our interactive-coding scheme overcomes these two issues, enabling the secure computation of any function over noisy networks while maintaining the communication complexity of the computation. Our analysis is the first to introduce multiparty interactive coding for randomized protocols in a noisy environment. We believe that our analysis can benefit other secure computation protocols and broader applications in distributed computation, particularly where randomization is adaptive, changing throughout the protocol rather than being fixed at the outset, and under different noise models.

As a first step, our result points to several interesting future directions: First, one can consider adversarial rather than stochastic noise. In the 2-party setting, it is known that adversarial noise can preclude correct and private computation [CPT13, GSW14]. Classifying the types of (adversarial) noise in which such computations are impossible remains a challenge and interesting future direction. We conjecture that resilience to mildly adversarial noise can be achieved through modest adaptations of our approach.

Another direction is extending the treatment to malicious adversaries. One can compile a malicious-secure protocol with ECC to achieve security in the presence of noisy channels. (Indeed, decoding into an arbitrary message does not introduce new attacks, thanks to the malicious security of the underlying protocol.) However, this is not the case with interactive coding, which presents significant challenges under malicious attacks, due to the need for synchronization mechanisms that involve rewinding and require careful analysis of success probabilities over randomized computations. As such, an extension to the malicious setting remains a challenging and, in our opinion, excellent research direction.

Acknowledgments

This research was supported in part by the United States-Israel Binational Science Foundation (BSF) through Grants No. 2020277. Additionally, Manuj Mukherjee was supported by the ANRF Startup Research Grant SRG/2023/00191.

References

- ABE⁺19. Noga Alon, Mark Braverman, Klim Efremenko, Ran Gelles, and Bernhard Haeupler. Reliable communication over highly connected noisy networks. *Distributed Computing*, 32(6):505–515, 2019.
- BCG⁺19. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 291–308, London, UK, November 11–15, 2019. ACM Press.
- Bea91. Donald Beaver. Foundations of secure interactive computing. In Joan Feigenbaum, editor, *Advances in Cryptology - CRYPTO '91, 11th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1991, Proceedings*, volume 576 of *Lecture Notes in Computer Science*, pages 377–391. Springer, 1991.
- BEGH16. Mark Braverman, Klim Efremenko, Ran Gelles, and Bernhard Haeupler. Constant-rate coding for multiparty interactive communication is impossible. In *Proceedings of the Forty-eighth Annual ACM Symposium on Theory of Computing*, STOC '16, pages 999–1010. ACM, 2016.
- BGMO17. Mark Braverman, Ran Gelles, Jieming Mao, and Rafail Ostrovsky. Coding for interactive communication correcting insertions and deletions. *IEEE Transactions on Information Theory*, 63(10):6256–6270, Oct 2017.
- BOGW88. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, STOC '88, page 1–10, New York, NY, USA, 1988. Association for Computing Machinery.
- Bra12. Mark Braverman. Towards deterministic tree code constructions. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ITCS '12, page 161–167, New York, NY, USA, 2012. Association for Computing Machinery.
- Can00. Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13(1):143–202, January 2000.
- CCXY18. Ignacio Cascudo, Ronald Cramer, Chaoping Xing, and Chen Yuan. Amortized complexity of information-theoretically secure MPC revisited. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 395–426, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- CDLR16. Ignacio Cascudo, Ivan Damgård, Felipe Lacerda, and Samuel Ranellucci. Oblivious transfer from any non-trivial elastic noisy channel via secret key agreement. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B: 14th Theory of Cryptography Conference, Part I*, volume 9985 of *Lecture Notes in Computer Science*, pages 204–234, Beijing, China, October 31 – November 3, 2016. Springer Berlin Heidelberg, Germany.
- CGH19. Keren Censor-Hillel, Ran Gelles, and Bernhard Haeupler. Making asynchronous distributed computations robust to noise. *Distributed Computing*, 32(5):405–421, Oct 2019.
- CK88a. C. Crepeau and J. Kilian. Achieving oblivious transfer using weakened security assumptions. In *Proceedings 1988/ 29th Annual Symposium on Foundations of Computer Science*, pages 42–52, 1988.
- CK88b. Claude Crépeau and Joe Kilian. Achieving oblivious transfer using weakened security assumptions (extended abstract). In *29th Annual Symposium on Foundations of Computer Science*, pages 42–52, White Plains, NY, USA, October 24–26, 1988. IEEE Computer Society Press.
- CLPS20. T-H. Hubert Chan, Zhibin Liang, Antigoni Polychroniadou, and Elaine Shi. Small memory robust simulation of client-server interactive protocols over oblivious noisy channels. In *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms*, SODA '20, pages 2349–2365, 2020.
- CPT13. Kai-Min Chung, Rafael Pass, and Sidharth Telang. Knowledge-preserving interactive coding. In *Proceedings of the 54th annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 449–458, 2013.
- Cré97. Claude Crépeau. Efficient cryptographic protocols based on noisy channels. In Walter Fumy, editor, *Advances in Cryptology - EUROCRYPT'97*, volume 1233 of *Lecture Notes in Computer Science*, pages 306–317, Konstanz, Germany, May 11–15, 1997. Springer Berlin Heidelberg, Germany.
- CS20. Gil Cohen and Shahar Samocha. Palette-Alternating Tree Codes. In Shubhangi Saraf, editor, *35th Computational Complexity Conference (CCC 2020)*, volume 169 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 11:1–11:29, Dagstuhl, Germany, 2020. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.

- CSSW06. Claude Crépeau, George Savvides, Christian Schaffner, and Jürg Wullschlegler. Information-theoretic conditions for two-party secure function evaluation. In Serge Vaudenay, editor, *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 538–554, St. Petersburg, Russia, May 28 – June 1, 2006. Springer Berlin Heidelberg, Germany.
- DFMS04. Ivan Damgård, Serge Fehr, Kirill Morozov, and Louis Salvail. Unfair noisy channels and oblivious transfer. In Moni Naor, editor, *TCC 2004: 1st Theory of Cryptography Conference*, volume 2951 of *Lecture Notes in Computer Science*, pages 355–373, Cambridge, MA, USA, February 19–21, 2004. Springer Berlin Heidelberg, Germany.
- EGL82. Shimon Even, Oded Goldreich, and Abraham Lempel. A randomized protocol for signing contracts. In David Chaum, Ronald L. Rivest, and Alan T. Sherman, editors, *Advances in Cryptology – CRYPTO’82*, pages 205–210, Santa Barbara, CA, USA, 1982. Plenum Press, New York, USA.
- FY92. Matthew K. Franklin and Moti Yung. Communication complexity of secure computation (extended abstract). In *24th Annual ACM Symposium on Theory of Computing*, pages 699–710, Victoria, BC, Canada, May 4–6, 1992. ACM Press.
- Gel17. Ran Gelles. Coding for interactive communication: A survey. *Foundations and Trends® in Theoretical Computer Science*, 13(1–2):1–157, 2017.
- GI05. V. Guruswami and P. Indyk. Linear-time encodable/decodable codes with near-optimal rate. *IEEE Trans. on Information Theory*, 51(10):3393–3400, 2005.
- GK19. Ran Gelles and Yael T. Kalai. Constant-rate interactive coding is impossible, even in constant-degree networks. *IEEE Transactions on Information Theory*, 65(6):3812–3829, 2019.
- GKR21. Ran Gelles, Yael Tauman Kalai, and Govind Ramnarayan. Efficient multiparty interactive coding—part I: Oblivious insertions, deletions and substitutions. *IEEE Transactions on Information Theory*, 67(6):3411–3437, 2021.
- GKR22. Ran Gelles, Yael Tauman Kalai, and Govind Ramnarayan. Efficient multiparty interactive coding—part II: Non-oblivious noise. *IEEE Transactions on Information Theory*, 68(7):4723–4749, 2022.
- GMW87. O. Goldreich, S. Micali, and A. Wigderson. How to play any mental game. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*, STOC ’87, page 218–229, New York, NY, USA, 1987. Association for Computing Machinery.
- Gol04. Oded Goldreich. *The Foundations of Cryptography - Volume 2: Basic Applications*. Cambridge University Press, 2004.
- GPZ18. Ran Gelles, Anat Paskin-Cherniavsky, and Vassilis Zikas. Secure two-party computation over unreliable channels. In *Security and Cryptography for Networks: 11th International Conference, SCN 2018, Amalfi, Italy, September 5–7, 2018, Proceedings*, page 445–463, Berlin, Heidelberg, 2018. Springer-Verlag.
- GSW14. Ran Gelles, Amit Sahai, and Akshay Wadia. Private interactive communication across an adversarial channel. In *Proceedings of the 5th Conference on Innovations in Theoretical Computer Science*, ITCS ’14, pages 135–144. ACM, 2014.
- Hae14. Bernhard Haeupler. Interactive channel capacity revisited. In *55th Annual Symposium on Foundations of Computer Science*, pages 226–235, Philadelphia, PA, USA, October 18–21, 2014. IEEE Computer Society Press.
- HIKN08. Danny Harnik, Yuval Ishai, Eyal Kushilevitz, and Jesper Buus Nielsen. Ot-combiners via secure computation. In Ran Canetti, editor, *Theory of Cryptography*, pages 393–411, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- HS16. William M. Hoza and Leonard J. Schulman. The adversarial noise threshold for distributed protocols. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 240–258, 2016.
- IKO⁺11. Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, Manoj Prabhakaran, Amit Sahai, and Jürg Wullschlegler. Constant-rate oblivious transfer from noisy channels. In Phillip Rogaway, editor, *Advances in Cryptology – CRYPTO 2011*, volume 6841 of *Lecture Notes in Computer Science*, pages 667–684, Santa Barbara, CA, USA, August 14–18, 2011. Springer Berlin Heidelberg, Germany.
- IPS08. Yuval Ishai, Manoj Prabhakaran, and Amit Sahai. Founding cryptography on oblivious transfer - efficiently. In David Wagner, editor, *Advances in Cryptology – CRYPTO 2008*, volume 5157 of *Lecture Notes in Computer Science*, pages 572–591, Santa Barbara, CA, USA, August 17–21, 2008. Springer Berlin Heidelberg, Germany.
- JKL15. Abhishek Jain, Yael Tauman Kalai, and Allison Lewko. Interactive coding for multiparty protocols. In *Proceedings of the 6th Conference on Innovations in Theoretical Computer Science*, ITCS ’15, pages 1–10, 2015.

- KMS16. Dakshita Khurana, Hemanta K. Maji, and Amit Sahai. Secure computation from elastic noisy channels. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 184–212, Vienna, Austria, May 8–12, 2016. Springer Berlin Heidelberg, Germany.
- LV15. Allison Lewko and Ellen Vitercik. Balancing communication for multi-party interactive coding, 2015.
- MG24. Manuj Mukherjee and Ran Gelles. Computation in server-assisted noisy networks. In *2024 IEEE International Symposium on Information Theory (ISIT)*, pages 3297–3301, 2024.
- PS21. Antigoni Polychroniadou and Yifan Song. Constant-overhead unconditionally secure multiparty computation over binary fields. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 812–841, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- Rab81. M. O. Rabin. How to exchange secrets with oblivious transfer. Technical Report TR-81, Aiken Computation Lab, Harvard University, 1981.
- Raj94. Sridhar Rajagopalan. *A Coding Theorem for Distributed Computation*. PhD thesis, University of California at Berkeley, Berkeley, CA, USA, 1994. UMI Order No. GAX95-29464.
- RS94. Sridhar Rajagopalan and Leonard Schulman. A coding theorem for distributed computation. In *STOC '94: Proceedings of the twenty-sixth annual ACM symposium on Theory of computing*, pages 790–799, 1994.
- Sch92. Leonard J. Schulman. Communication on noisy channels: a coding theorem for computation. *Foundations of Computer Science, Annual IEEE Symposium on*, pages 724–733, 1992.
- Sch93. Leonard J. Schulman. Deterministic coding for interactive communication. In *STOC '93: Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*, pages 747–756, New York, NY, USA, 1993. ACM.
- Sha48. Claude E. Shannon. A mathematical theory of communication. *ACM SIGMOBILE Mobile Computing and Communications Review*, 5(1):3–55, 2001. Originally appeared in *Bell System Tech. J.* 27:379–423, 623–656, 1948.
- Spi95. Daniel A. Spielman. Linear-time encodable and decodable error-correcting codes. In *Proceedings of the twenty-seventh annual ACM symposium on Theory of computing*, STOC '95, pages 388–397, New York, NY, USA, 1995. ACM.
- Wul09. Jürg Wullschlegel. Oblivious transfer from weak noisy channels. In Omer Reingold, editor, *TCC 2009: 6th Theory of Cryptography Conference*, volume 5444 of *Lecture Notes in Computer Science*, pages 332–349. Springer Berlin Heidelberg, Germany, March 15–17, 2009.
- Yao82. Andrew C. Yao. Protocols for secure computations. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 160–164, 1982.

A Missing Preliminaries

Definition 3 (Statistical Distance). We define $SD(U, V)$ as the statistical distance between distributions U and V .

$$SD(U, V) = \frac{1}{2} \sum_{\omega} |U(\omega) - V(\omega)|.$$

For two distributions U and V we say that they are ε -close, and denote $U \approx_{\varepsilon} V$ if $SD(U, V) \leq \varepsilon$.

The notion of ε -closeness is easily extendible to probability ensembles (i.e., families of distributions) $U = \{U_{\kappa}\}_{\kappa}$ and $V = \{V_{\kappa}\}_{\kappa}$ parameterized by a security parameter κ .

Definition 4 (Protocol with Correlations). An n -party protocol π with $\mathbb{C}$ correlations for evaluating the functionality f in $|\pi|$ rounds includes a set of next message functions $\pi = (\pi_1, \dots, \pi_n)$ and a correlation index t . At the start of each round r , party P_i executes the next message function π_i that takes as input some x_i , correlation instances $\text{corr}^i[: r/t]$ and the set m_j^r for all $j \neq i$ which denote the set of messages received by party i from party j up to r rounds and computes the next message to be sent to each party. The output of the next message function in the $|\pi|$ round represents the output of the corresponding parties. We denote $\pi_i(x_i, \text{corr}^i)$ to represent output of party P_i .

We say protocol π is **correct**, if for any inputs $x_1, x_2, \dots, x_n$ and for any correlation $\text{corr} \in \mathbb{C}^{|\pi|/t}$, it holds $(\pi_1(x_1, \text{corr}^1), \dots, \pi_n(x_n, \text{corr}^n)) = f(x_1, x_2, \dots, x_n)$

A.1 IC Preliminaries

Lemma 2 (Shannon Coding Theorem [Sha48]). For any discrete memoryless channel Ch with capacity $\mathcal{C}$ and any k , there exists a code $\text{ECC} : \{0, 1\}^k \rightarrow \{0, 1\}^n$ and $\text{ECC}^{-1} : \{0, 1\}^n \rightarrow \{0, 1\}^k$ with $n = O(\frac{1}{\mathcal{C}}k)$ such that for any $m \in \{0, 1\}^k$ it holds that,

$$\Pr \left[\text{ECC}^{-1}(\text{Ch}(\text{ECC}(m))) \neq m \right] < 2^{-\Omega(n)}.$$

Recall that the capacity of the BSC_{ε} channel is

$$\mathcal{C}_{\text{BSC}_{\varepsilon}} = 1 + \varepsilon \log \varepsilon + (1 - \varepsilon) \log(1 - \varepsilon) = O_{\varepsilon}(1)$$

and that efficient constructions of such codes are well known (e.g., [Spi95, GI05]).

Outline of RS Scheme We outline the multiparty interactive coding scheme presented in [RS94]. Each party i maintains a list of messages sent to party j for all $j \neq i$. In each iteration of the scheme, party i decodes the messages sent by each party $j \neq i$. The messages are symbols sent by party j to party i encoded using a tree code. The messages are either the next bit to be sent according to the underlying protocol or a special $\leftarrow$ symbol. After decoding the messages, party i parses the list of messages received from each party j by removing each $\leftarrow$ symbol and the previous undeleted message from the $\leftarrow$. Then, party i checks if the messages sent in the previous rounds are consistent with those received. Note that due to noise, the tree code decodings may be incorrect, which might result in incorrect messages sent by party i or might have inadvertently deleted correctly received messages. If this consistency check is passed, party i sends the next bit to each party j via the tree code; otherwise, it encodes the $\leftarrow$ symbol with the tree code and sends it to each party.

The tree code symbols are again encoded using an ECC length $O(d+1)$ where d is the maximum degree of the vertices in the communication graph. This leads to a rate of $O_\epsilon\left(\frac{1}{d+1}\right)$. To achieve constant rate, Alon et al. use a so-called A2A protocol that performs all-to-all communication among n parties in a constant number of rounds with error probability $2^{-\Omega_\epsilon(\sqrt{n})}$.

A2A Protocol The A2A protocol from [ABE⁺19] solves the following task. Each party P_i has a vector $b_i = \{b_{i1}, \dots, b_{in}\}$ of n bits where it wants to send b_{ij} to party j . The output of party P_i is a vector $c_i = \{b_{1i}, \dots, b_{ni}\}$.

The input of the A2A protocol can be represented as a matrix A of size $n \times n$. The contents of the matrix are $A[i][j] = b_{ij}$, where party i wants to send b_{ij} to party j . At the start of the protocol, party i knows the bits in the i -th row. At the end of the protocol, party i knows the bits of the i -th column.

The basic idea of the protocol is as follows. The matrix A can be divided into n submatrices $B_{t,l}$ of size $k \times k$ where $k = \sqrt{n}$. Each $B_{t,l}$ is assigned to a party $(t-1)k + l$. The communication happens in two phases. In the first phase of communication, each party i takes the k bits of $B_{t,l}$ that it knows, encodes them with an ECC to get $k' = O_\epsilon(k)$ bits, splits the k' bits into chunks of k bits and sends the j^{th} chunk to party $(ki + j) \bmod n$, which relays the chunk to party $(t-1)k + l$. At the end of the first communication phase, each party knows the bits of its submatrix. In the second phase, the transpose direction happens. Each party $(t-1)k + l$ takes the k bits of $B_{t,l}$ that belong to party i , encodes them with ECC to get k' bits, splits the k' bits into chunks of k bits and sends the j^{th} chunk to party $(i + kj) \bmod n$ who relays the chunk to party i . At the end of the second phase, each party knows the bits it is supposed to learn. The ECC fails with probability $1 - 2^{-\Omega(\sqrt{n})}$.

The probability of failure of the A2A protocol is $1 - 2^{-\Omega(\sqrt{n})}$, which is based on the failure probability of ECC and the BSC channel error probability.

Algorithm A2A

A Resilient all-to-all (A2A) Communication [ABE⁺19]

- **Inputs:** party i for $i \in [n]$ has a single bit $b_{i,j}$ to communicate to j for all $j \in [n]$.
- **Outputs:** party i outputs all the bits address to it, $(b_{1,i}, \dots, b_{n,i})$.
- **Assume:** error correcting code $\text{ECC} : \{0, 1\}^k \rightarrow \{0, 1\}^{k'}$ with $k' = O(k)$ and constant distance (see Lemma 2)
- **Algorithm:**
 1. Write the n^2 bits $\{b_{i,j}\}$ as the matrix $A \in \{0, 1\}^{n \times n}$. Set $k = \sqrt{n}$.
 2. Re-write A as n disjoint sub-matrices $B_{t,l} \in \{0, 1\}^{k \times k}$,

$$A = \begin{pmatrix} B_{1,1} & B_{1,2} & \cdots & B_{1,k} \\ B_{2,1} & B_{2,2} & \cdots & B_{2,k} \\ & & \ddots & \\ B_{k,1} & B_{k,2} & \cdots & B_{k,k} \end{pmatrix},$$

with

$$B_{t,l} = \begin{pmatrix} a_{(t-1)k+1,(l-1)k+1} & a_{(t-1)k+1,(l-1)k+2} & \cdots & a_{(t-1)k+1,lk} \\ a_{(t-1)k+2,(l-1)k+1} & a_{(t-1)k+2,(l-1)k+2} & \cdots & a_{(t-1)k+2,lk} \\ \vdots & & \ddots & \vdots \\ a_{tk,(l-1)k+1} & a_{tk,(l-1)k+2} & \cdots & a_{tk,lk} \end{pmatrix}.$$

3. Associate each $B_{t,l}$ with a unique party, say, $k(t-1) + l$.
4. For any t, l , party i encodes the k bits that belong to $B_{t,l}$ with an ECC to obtain the k' bits $\mathbf{a} = a_1, \dots, a_{k'}$.
5. Party i splits $\mathbf{a}$ into k disjoint parts of size $O(1)$ bits each and sends the j -th part to party $(ki + j) \bmod n$ in $O(1)$ rounds. Then, each party relays the message to party $k(t-1) + l$ in $O(1)$ rounds.
6. For any t, l , party $k(t-1) + l$ encodes the i^{th} column of $B_{t,l}$ that is to be sent to party $(l-1)k + i$ with an ECC to obtain the k' bits $\mathbf{d} = d_1, \dots, d_{k'}$.
7. Party $k(t-1) + l$ splits $\mathbf{d}$ into k disjoint parts of size $O(1)$ bits each and sends the j -th part to party $(i + kj) \bmod n$ in $O(1)$ rounds. Then, each party relays the message to party $(l-1)k + i$ in $O(1)$ rounds.

Fig. 7: A Resilient all-to-all (A2A) Communication

Tree Codes A *tree code* [Sch93] is an encoding technique for strings of d -sized alphabets. Informally, a d -ary tree code T is an infinite d -ary tree, where each edge of the tree is labeled with an alphabet Γ , and each node has d children. We say that the tree code has distance δ if, for any two different paths of length h (representing encodings of two strings of equal length), the paths differ by at least δh edges. Formally,

Definition 5 (Tree Codes). A tree code with input alphabet S , output alphabet Σ , and distance α is an online encoding algorithm $\mathcal{T} : S^* \rightarrow \Sigma^*$ that satisfies the following:

1. (online) For any $s \in S^*$, $s = s[1]s[2] \dots s[k]$, it holds that $\mathcal{T}(s[1]s[2] \dots s[k]) = \mathcal{T}(s[1]s[2] \dots s[k-1]) \circ \gamma_k$.
2. (distance) For any $s, h_1, h_2 \in S^*$ such that $h_1[1] \neq h_2[1]$ and $|h_1| = |h_2|$, it holds that $d(\mathcal{T}(s \circ h_1), \mathcal{T}(s \circ h_2)) > \alpha|h_1|$,

where $d(\cdot, \cdot)$ indicate the Hamming distance between two strings.

Tree codes of constant distance (and constant output alphabet with respect to the input alphabet and the distance) are known to exist, see [Sch93, Bra12, BGMO17, CS20]; see also [Gel17].

Lemma 3 ([Sch93]). For any $\alpha < 1$ and input alphabet S , there exists a tree code of infinite depth and distance α whose output alphabet is of size $|\Sigma| = O(|S|)^{1/(1-\alpha)}$.

Tree codes are decoded using *minimum distance decoding*. More precisely, the decoding function $\mathcal{T}^{-1} : \Sigma^* \rightarrow S^*$ outputs the string whose encoding minimizes the hamming distance to the input,

$$\mathcal{T}^{-1}(r) \triangleq \operatorname{argmin}_{s \in S^{|r|}} d(r, \mathcal{T}(s)).$$

Suppose a message $s \in S^*$ is tree coded to obtain $\sigma = \mathcal{T}(s)$, and then sent over some channel (say BSC_ε after converting the symbols from Σ into $\log |\Sigma|$ bits). Due to channel corruptions, referred to as *channel errors*, some $\sigma' \in \Sigma^*$ is received at the output of the channel. We define $d(\sigma, \sigma')$ to be the *number* of channel errors. We say that a *tree code decoding error* occurs if $s' := \mathcal{T}^{-1}(\sigma') \neq s$. The *length* of the tree code error is defined as the length of the longest suffix in error. More precisely, the length of the tree code error is said to be h if $s'[1, |s| - h] = s[1, |s| - h]$ and $s'[|s| - h + 1, |s|] \neq s[|s| - h + 1, |s|]$.

The following lemma relates the length of a tree code error with the number of channel errors in the suffix of the tree-coded message.

Lemma 4 (Lemma 2.4 in [Gel17]). *Let $\mathcal{T} : S^* \rightarrow \Sigma^*$ be a tree code of distance of distance α , and let $s \in S^*$. Suppose $\sigma = \mathcal{T}(s)$ is sent over some channel which outputs $\sigma' \in \Sigma^*$. Then a tree code error of length h occurs only if $d(\sigma[|s| - h + 1, |s|], \sigma'[|s| - h + 1, |s|]) \geq \frac{h\alpha}{2}$.*

B Analysis of the ChunkRS scheme

In this section we prove Theorem 5 showing that the ChunkRS scheme leads to a successful simulation of any protocol π with high probability.

We begin by defining the progress made by a party in ChunkRS to be the number of chunks of π it has successfully simulated. More precisely, the progress made by p_i after t iterations (chunks), denoted $\text{RP}(p_i, t)$, is defined as

$$\text{RP}(p_i, t) \triangleq \arg\max_{l \in [t]} \left\{ z_i(t)[1, l] = m_{\rightarrow i}(l \cdot c) \text{ and } \sigma_i(t)[1, l] = m_{i \rightarrow}(l \cdot c) \right\}, \quad (1)$$

when the equality refers to the respective induced bit-strings. Note that we use $z_i(t)$ and $\sigma_i(t)$ as these correspond to the parsed versions of the transcripts R_i and T_i after they are updated in the t -th iteration, treated as strings over the set S (where every symbol is $\{0, 1\}^c$, i.e., c -bits). Also recall that $m_{i \rightarrow}(l \cdot c)$ and $m_{\rightarrow i}(l \cdot c)$ are the correct sent and received transcripts of p_i in π up to round $l \cdot c$ (of π), which corresponds to l chunks of c bits. Recalling that $k = \frac{2|\pi|}{c}$, the simulation ChunkRS will be successful if $\text{RP}(p_i, k) \geq \frac{k}{2}$, at all parties p_i .

To analyze the progress, we define a *historical path*⁷ of length t , as a sequence of t parties, $\phi \in (\{p_i\}_{i \in [n]})^t$. The sequence may contain the same party multiple times, i.e., it is possible that $\phi[t'] = \phi[t'']$ for any $1 \leq t' < t'' \leq t$. The set of all historical paths of length t is denoted $\mathcal{H}_t$.

Next we define three more quantities. For any $i \in [n]$, define $B(p_i, t)$ to be the number of iterations up to and including t , where BK symbols were sent by p_i . It immediately follows from (1) and the definition of the ParseChunk operation that

$$\text{RP}(p_i, t) \leq t - 2B(p_i, t). \quad (2)$$

We say that a *SimChunk decoding error* is made at iteration l if there exists some party p_i which made a decoding error in the A2A algorithm in one of the c rounds of SimChunk at iteration l . Define $D(t)$ to be the maximum number of SimChunk decoding errors across all historical paths in $\mathcal{H}_t$. More precisely,

$$D(t) \triangleq \max_{\phi \in \mathcal{H}_t} \sum_{l=1}^t \mathbf{1}\{\phi(l) \text{ made a SimChunk decoding error at iteration } l\}.$$

⁷ Here, we divert from the analysis of [RS94], where the historical path is defined for each p_i separately. The reason for this change will become apparent in the analysis of Lemma 5, where the subcase ‘propagated error’ of [RS94] needs to be broken into two separate sub-cases for our analysis.

We say that a party p_i makes a tree code decoding error in iteration l if it makes a tree code decoding error for the string of symbols sent to it by some party p_j , i.e., $T_{j,i} \neq w_{i,j}$.

For any $\phi \in \mathcal{H}_t$, let $E(\phi)$ count the instances where party $\phi[l]$ made a tree code decoding error, that is,

$$E(\phi) \triangleq \sum_{l=1}^{|\phi|} \mathbf{1}\{\phi[l] \text{ made a tree code decoding error in iteration } l\}.$$

Also, we say that a tree code error *began* at $\phi[l]$, if the party $\phi[l]$ did not make any tree code decoding error in iteration $l-1$, but it made at least one tree code decoding error in iteration l . We say that $\phi \in \mathcal{H}_t$ has *disjoint tree errors* if for any l for which $\phi[l] \neq \phi[l-1]$, the party $\phi[l]$ makes a tree code decoding error at iteration l only if that tree code error began at $\phi[l]$.

B.1 Proof of Theorem 5

The proof of Theorem 5 follows from the three subsequent lemmas. The first of these lemmas relate progress made by p_i up to iteration t with the number of BKs sent, the number of iterations with decoding errors in **SimChunk**, and the number of tree code decoding errors.

Lemma 5. *Given an execution of Algorithm 1, and for any iteration t and any party p_i , there exists a historical path $\phi_{i,t} \in \mathcal{H}_t$ with disjoint tree errors satisfying*

$$t \leq RP(p_i, t) + B(p_i, t) + D(t) + E(\phi_{i,t}). \quad (3)$$

As a corollary to Lemma 5, we obtain the following lemma.

Lemma 6. *Given an execution of Algorithm 1, and for any iteration t and any party p_i , there exists a historical path $\phi_{i,t} \in \mathcal{H}_t$ with disjoint tree errors, such that*

$$E(\phi_{i,t}) + D(t) \geq \frac{t - RP(p_i, t)}{2}.$$

Proof. Using Lemma 5, there exists $\phi_{i,t} \in \mathcal{H}_t$ with disjoint tree errors, satisfying

$$\begin{aligned} 2t &\leq 2RP(p_i, t) + 2B(p_i, t) + 2(D(t) + E(\phi_{i,t})) \\ &\leq RP(p_i, t) + t + 2(D(t) + E(\phi_{i,t})), \end{aligned}$$

where the second inequality follows from Eq. (2). Lemma 6 then follows by rearranging the above. $\square$

The third and final lemma is obtained as a corollary to Lemma 6, and is stated below.

Lemma 7. *Following an execution of Algorithm 1, the probability that there exists a party p_i with $RP(p_i, k) < \frac{k}{2}$ is $2^{-\Omega(k\sqrt{n})}$.*

Recalling that **ChunkRS** leads to successful simulation if $RP(p_i, k) \geq \frac{k}{2}$ for all $i \in [n]$, Lemma 7 ensures that the probability of failure of **ChunkRS** is $o(1)$. Finally, noting that each iteration of **SimChunk** involves $O_\varepsilon(c)$ rounds of communication, the round complexity of **ChunkRS**, which contains $k = O(|\pi|/c)$ iterations, is $O_\varepsilon(|\pi|)$. Therefore, this completes the proof of Theorem 5 modulo the proofs of Lemmas 5 and 7. These are produced in the subsequent subsections.

We begin with the proofs of Lemmas 5 and 7.

Proof of Lemma 5 To start the proof, we classify the action of p_i in iteration t into six mutually exclusive and exhaustive categories listed below.

- Progress (**p**): Occurs if $\text{RP}(p_i, t) = \text{RP}(p_i, t - 1) + 1$.
- Necessary backup (**bk**): If BK was sent in iteration t and $\text{RP}(p_i, t) = \text{RP}(p_i, t - 1)$.
- Harmful tree error (**hte**): When neither **p** or **bk** occurs, and a tree code decoding error is made by p_i .
- Harmful backup (**hbk**): There was no tree code decoding error, but BK was sent and it deleted a correct chunk, i.e., $\text{RP}(p_i, t) = \text{RP}(p_i, t - 1) - 1$.
- Propagated error with a tree code error (**pete**): There was no tree code decoding error by p_i , BK was not sent by p_i , a tree code decoding error was made by some other party $p_j, j \neq i$ in iteration t , and $\text{RP}(p_i, t - 1) = \text{RP}(p_i, t)$.
- Propagated error with **SimChunk** decoding error (**pede**): Similar to **pete** except no party made a tree code decoding error in iteration t .

We note here that these categories are similar to those in the progress proof of [RS94], except for **pede** and **pete**, which were merged into a single category in [RS94]. It is exactly in the analysis of these categories we see the effect of the chunked simulation, where errors due to the free simulation of a chunk in the **SimChunk** procedure (Algorithm 2) need to be accounted for.

We shall prove the result by induction. Furthermore, we shall assume in our induction hypothesis that for any $l \leq t - 1$, and any party p_i , the historical path $\phi_{i,l}$ satisfying the induction hypothesis has $\phi_{i,l}[l] = p_i$ unless p_i had **pete** at iteration l .

To proceed, note that the induction hypothesis obviously holds when $t = 0$. We assume that the result holds for all parties upto iteration $t - 1$. To complete the proof, we fix a party p_i , and show that the hypothesis of Lemma 5 holds for iteration t , in a case by case basis.

To begin with, observe that by definition $D(t - 1) \leq D(t)$. Second, note that if $\phi_{i,t} = \phi_{i,t-1} \circ p_i$, then $E(\phi_{i,t}) \geq E(\phi_{i,t-1})$. Furthermore, we claim that $\phi_{i,t}$ obtained in this fashion will have disjoint tree errors. To see this, observe that by the induction hypothesis, $\phi_{i,t-1}$ has disjoint tree errors. Now, if $\phi_{i,t-1}(t - 1) = p_i$, then by default $\phi_{i,t}$ will have disjoint tree errors. On the other hand, if $\phi_{i,t-1}(t - 1) \neq p_i$, then by the induction hypothesis, p_i must have **pete** in iteration $t - 1$. Thus, by definition of **pete**, p_i made no tree code decoding error in $t - 1$. Hence, even if p_i made a tree code decoding error at t , that tree code error began at t . So, $\phi_{i,t}$ will have disjoint tree errors.

We first do away with the easy cases. If **p** occurred then BK couldn't have been sent by p_i in iteration t . Hence $B(p_i, t - 1) = B(p_i, t)$. Also, define $\phi_{i,t} = \phi_{i,t-1} \circ p_i$. Then, the induction hypothesis holds for p_i at t , by using the induction hypothesis at $t - 1$ and $\text{RP}(p_i, t) = \text{RP}(p_i, t - 1) + 1$. Next, assume **bk** occurred. Hence $B(p_i, t) = B(p_i, t - 1) + 1$ and $\text{RP}(p_i, t) = \text{RP}(p_i, t - 1)$. Again, define $\phi_{i,t} = \phi_{i,t-1} \circ p_i$. Therefore, the induction hypothesis at t holds by using the hypothesis at $t - 1$.

Next, assume **hte** has occurred, and define $\phi_{i,t} = \phi_{i,t-1} \circ p_i$. Hence, we must have $E(\phi_{i,t}) = E(\phi_{i,t-1}) + 1$, as **hte** implies p_i making a tree code decoding error at t . Next, note that among the remaining three quantities in the right hand side of the induction hypothesis, only $\text{RP}(p_i, t)$ is not monotonic in t . Now, if $\text{RP}(p_i, t) = \text{RP}(p_i, t - 1)$, then the induction hypothesis holds using $E(\phi_{i,t}) = E(\phi_{i,t-1}) + 1$. Otherwise, if $\text{RP}(p_i, t) = \text{RP}(p_i, t - 1) - 1$, then it must be the case that a BK was sent by p_i in iteration t , and hence $B(p_i, t) = B(p_i, t - 1) + 1$. Thus, in conjunction with $E(\phi_{i,t}) = E(\phi_{i,t-1}) + 1$, this proves the induction hypothesis at t .

We now move to the three remaining cases which require slightly more involved analysis. Before beginning the analysis, note that if for any $j \neq i$, if $\phi_{i,t} = \phi_{j,t-1} \circ p_i$, then obviously $E(\phi_{i,t}) \geq E(\phi_{j,t-1})$. Furthermore, since in the remaining cases p_i doesn't make any tree code decoding error

at t , and $\phi_{j,t-1}$ has disjoint tree errors by the induction hypothesis, therefore, any $\phi_{i,t}$ obtained in the above fashion also has disjoint tree errors.

To proceed, we consider that **hbk** has occurred. Hence, we must have $B(p_i, t) = B(p_i, t-1) + 1$ and $RP(p_i, t) = RP(p_i, t-1) - 1$. Now, since **BK** was sent by p_i , there must have been a consistency check failure (i.e., $\text{flag} = 0$ in this iteration). We first note that no tree code decoding error was made by p_i at iteration t . Hence, for all $j \neq i$, the variable $\tilde{z}_{i,j}(t)$ is exactly the parsed version of the string of symbols sent by p_j to p_i so far.

We now divide the analysis into two subcases. First we assume that there was some $j^* \neq i$ such that $|\tilde{z}_{i,j^*}(t)| < |z_{i,j^*}(t)|$, and this led to the consistency check failure. This means that $B(p_{j^*}, t-1) \geq B(p_i, t-1) + 1$. Moreover, since sending a **BK** symbol in t resulted in $RP(p_i, t)$ decreasing by 1, we must have $RP(p_i, t-1) = t-1 - 2B(p_i, t-1)$. Thus, using (2), we get

$$\begin{aligned} RP(p_{j^*}, t-1) &\leq t-1 - 2B(p_{j^*}, t-1) \\ &\stackrel{(a)}{=} RP(p_i, t-1) + 2B(p_i, t-1) - 2B(p_{j^*}, t-1) \\ &\stackrel{(b)}{\leq} RP(p_i, t-1) + B(p_i, t-1) - B(p_{j^*}, t-1) - 1, \end{aligned} \quad (4)$$

where (a) uses $RP(p_i, t-1) = t-1 - 2B(p_i, t-1)$, and (b) follows from $B(p_{j^*}, t-1) \geq B(p_i, t-1) + 1$. Hence, using the induction hypothesis at p_{j^*} at $t-1$, and (4), and by defining $\phi_{i,t} = \phi_{j^*,t-1} \circ p_i$, we have

$$\begin{aligned} t-1 &\leq RP(p_j, t-1) + B(p_j, t-1) + D(t-1) + E(\phi_{j^*,t-1}) \\ &\leq RP(p_i, t-1) + B(p_i, t-1) - 1 + D(t) + E(\phi_{i,t}) \\ &= RP(p_i, t) + B(p_i, t) - 1 + D(t) + E(t), \end{aligned}$$

where the last equality follows using $RP(p_i, t) = RP(p_i, t-1) - 1$ and $B(p_i, t) = B(p_i, t-1) + 1$. Hence, we have $t \leq RP(p_i, t) + B(p_i, t) + D(t) + E(\phi_{i,t})$, with $\phi_{i,t}$ having disjoint tree errors as required.

Next, assume that the other sub-case of **hbk** occurred, i.e., for all $j \neq i$, $|\tilde{z}_{i,j}(t)| \geq |z_{i,j}(t)|$. This immediately implies that for all $j \neq i$, we must have $B(p_i, t-1) \geq B(p_j, t-1)$. Next, the failure of consistency check implies that there must be at least one p_{j^*} for which $\tilde{z}_{i,j^*}(t)$ is not consistent with π_0 . This is because $z_{i,j^*}^{(t)}$ must be completely consistent π_0 , as otherwise $RP(p_i, t-1)$ would not have decreased by 1. Now, since p_i did not commit any tree code decoding error, $\tilde{z}_{i,j}(t)$, for all $j \neq i$, correspond exactly to the parsed version of p_j 's sent transcript to p_i . Thus, consistency check failure implies that the number of chunks of $\tilde{z}_{i,j^*}(t)$ consistent with π is strictly fewer than $|z_{i,j^*}(t)|$. Hence, we have $RP(p_i, t-1) \geq RP(p_{j^*}, t-1) + 1$. Plugging this and $B(p_i, t-1) \geq B(p_{j^*}, t-1)$ in the induction hypothesis for p_j at $t-1$, and defining $\phi_{i,t} = \phi_{j^*,t-1} \circ p_i$, we get

$$\begin{aligned} t-1 &\leq RP(p_{j^*}, t-1) + B(p_{j^*}, t-1) + D(t-1) + E(\phi_{j^*,t-1}) \\ &\leq RP(p_i, t-1) - 1 + B(p_i, t-1) + D(t) + E(\phi_{i,t}) \\ &= RP(p_i, t) + B(p_i, t) - 1 + D(t) + E(\phi_{i,t}), \end{aligned}$$

where the equality follows using $RP(p_i, t) = RP(p_i, t-1) - 1$ and $B(p_i, t) = B(p_i, t-1) + 1$. Hence, we have $t \leq RP(p_i, t) + B(p_i, t) + D(t) + E(\phi_{i,t})$, with $\phi_{i,t}$ having disjoint tree errors as required.

Next, we verify the induction hypothesis if **pede** occurred. In this case, note that no tree code decoding error by any party for iteration t implies that everyone has the correct version of everyone

else's transcript. In particular, this means $\tilde{z}_{i,j}(t) = \sigma_{j,i}(t)$ for all $j \neq i$. Now, since consistency check was passed, we must have $|\tilde{z}_{i,j}(t)| \geq |z_{i,j}(t)|$ for all $j \neq i$. This implies $B(p_i, t-1) \geq B(p_j, t-1)$ for all $j \neq i$.

Firstly, we prove that $RP(p_j, t-1) \geq RP(p_i, t-1) + 1$ for all $j \neq i$ cannot occur. Indeed, if $RP(p_j, t-1) \geq RP(p_i, t-1) + 1$ for all $j \neq i$, then due to the fact that $\tilde{z}_{i,j}(t) = \sigma_{j,i}(t)$ for all $j \neq i$, the **SimChunk** procedure (Algorithm 2) would ensure that $RP(p_i, t) = RP(p_i, t-1) + 1$. We now break the analysis into a pair of subcases. The first subcase assumes that there exists p_{j^*} satisfying $RP(p_i, t-1) \geq RP(p_{j^*}, t-1) + 1$. Then, using the induction hypothesis at $t-1$ for p_{j^*} , and setting $\phi_{i,t} = \phi_{j^*,t-1} \circ p_i$, we get

$$\begin{aligned} t-1 &\leq RP(p_{j^*}, t-1) + B(p_{j^*}, t-1) + D(t-1) + E(\phi_{j^*,t-1}) \\ &\stackrel{(a)}{\leq} RP(p_i, t-1) - 1 + B(p_i, t-1) + D(t) + E(\phi_{i,t}) \\ &\stackrel{(b)}{=} RP(p_i, t) - 1 + B(p_i, t) + D(t) + E(\phi_{i,t}), \end{aligned}$$

where (a) follows by recalling $B(p_i, t-1) \geq B(p_{j^*}, t-1)$, and (b) uses $RP(p_i, t-1) = RP(p_i, t)$ and $B(p_i, t-1) = B(p_i, t)$.

If the above subcase doesn't occur, then the set $A = \{p_j : RP(p_j, t-1) = RP(p_i, t-1), j \neq i\}$ is non-empty. Since, there was no tree code decoding error at any of the parties, all parties $p_j \in A$ can pass the consistency check only if $RP(p_j, t-1) = |\sigma_{j,i}(t)|$. Therefore, the only way that p_i could make no progress in this situation would be due to the fact that some party in A , or p_i itself, made a decoding error during **SimChunk**. Let that party be j^* , and define $\phi_{i,t} = \phi_{j^*,t-1} \circ p_i$. Now, the decoding error in **SimChunk** implies $D(t) = D(t-1) + 1$. This gives us the induction hypothesis at t via the hypothesis at $t-1$, as well as $RP(p_i, t) = RP(p_i, t-1)$ and $B(p_i, t) = B(p_i, t-1)$.

Alternatively, it may be the case that some party in A failed the consistency check. Let that party be p_{j^*} . Since, every party must have at least as much progress as p_i , we must have that $RP(p_{j^*}, t-1) < t-1 - 2B(p_{j^*}, t-1)$. On the other hand, since p_i still passed the consistency check, we still have $RP(p_i, t-1) = t-1 - 2B(p_i, t-1)$. Hence, we must have $B(p_{j^*}, t-1) \leq B(p_i, t-1) - 1$. Thus, defining $\phi_{i,t} = \phi_{j^*,t-1} \circ p_i$, and using the induction hypothesis at p_{j^*} at iteration $t-1$, we have

$$\begin{aligned} t-1 &\leq RP(p_{j^*}, t-1) + B(p_{j^*}, t-1) + E(\phi_{j^*,t-1}) + D(t-1) \\ &\leq RP(p_i, t-1) + B(p_i, t-1) - 1 + E(\phi_{i,t}) + D(t) \\ &= RP(p_i, t) + B(p_i, t) - 1 + E(\phi_{i,t}) + D(t), \end{aligned}$$

where the last step follows by recalling $RP(p_i, t-1) = RP(p_i, t)$ and $B(p_i, t-1) = B(p_i, t)$. This completes the proof of the induction hypothesis when **pede** occurred for p_i at t .

To finish the proof, we need to consider the last case where **pete** occurs for p_i at t . Arguing exactly similar to the **pede** scenario, we have that $B(p_j, t-1) \leq B(p_i, t-1)$ and $RP(p_j, t-1) \geq RP(p_i, t-1)$ for all $j \neq i$. Now, define the sets $A = \{p_j : RP(p_j, t-1) = RP(p_i, t-1), j \neq i\}$ and $B = \{p_j : RP(p_j, t-1) > RP(p_i, t-1)\}$. Now, we divide the analysis into two subcases.

First, consider that $A = \emptyset$. However, as argued in the **pede** case, $B \subsetneq [n] \setminus \{i\}$. Hence, there must exist some j^* satisfying $RP(p_{j^*}, t-1) \leq RP(p_i, t-1) - 1$. Arguing as in the equivalent case for **pede** and setting $\phi_{i,t} = \phi_{j^*,t-1} \circ p_i$, we obtain the induction hypothesis at t . Thus, for the remaining subcases, we assume that $A \neq \emptyset$.

For the second subcase, we assume that there exists $j^* \in A$ that made the tree code decoding error at t . Define $\phi_{i,t} = \phi_{j^*,t-1} \circ p_{j^*}$, and observe that $E(\phi_{i,t}) = E(\phi_{j^*,t-1}) + 1$. Again, as argued

previously for $\phi_{i,t-1} \circ p_i$, we can argue that $\phi_{j^*,t-1} \circ p_{j^*}$ has disjoint tree errors due to the induction hypothesis. Also, note here that $j^* \neq i$ is being added to get $\phi_{i,t}$, but this is occurring when **pete** occurred, as required by the induction hypothesis. Now, using the induction hypothesis for j^* at $t - 1$, we get

$$\begin{aligned} t - 1 &\leq \text{RP}(p_{j^*}, t - 1) + \text{B}(p_{j^*}, t - 1) + \text{D}(t - 1) + \text{E}(\phi_{j^*,t-1}) \\ &\stackrel{(a)}{\leq} \text{RP}(p_i, t - 1) + \text{B}(p_i, t - 1) + \text{D}(t) + \text{E}(\phi_{i,t}) - 1 \\ &\stackrel{(b)}{=} \text{RP}(p_i, t) + \text{B}(p_i, t) + \text{D}(t) + \text{E}(\phi_{i,t}) - 1 \end{aligned}$$

where (a) follows by recalling $j^* \in A$, $\text{B}(p_i, t - 1) \geq \text{B}(p_{j^*}, t - 1)$, and $\text{E}(\phi_{i,t}) = \text{E}(\phi_{j^*,t-1}) + 1$, and (b) follows by recalling $\text{B}(p_i, t - 1) = \text{B}(p_i, t)$ and $\text{RP}(p_i, t - 1) = \text{RP}(p_i, t)$ due to **pete**. Rearranging (b) we obtain the induction hypothesis for t .

To finish the proof of Lemma 5, we look at the final subcase where no party in A made a tree code decoding error. Like the **pede** case before, we can again argue in this case that some party p_{j^*} with $j^* \in A$, or p_i itself must have made a decoding error in **SimChunk**, or some p_{j^*} with $j^* \in A$ failed the consistency check. In both cases, we define $\phi_{i,t} = \phi_{j^*,t-1} \circ p_i$, and argue exactly similar to the **pede** case to prove the induction hypothesis.

Proof of Lemma 7 We begin by fixing a party p_i . If $\text{RP}(p_i, k) < \frac{k}{2}$, then, by Lemma 6, there exists a history path $\phi_{i,k}$ with disjoint tree errors satisfying $\text{E}(\phi_{i,k}) + \text{D}(k) > \frac{k}{4}$. We begin by upper bounding the probability of this event.

Now, note that if $\text{E}(\phi_{i,k}) + \text{D}(k) > \frac{k}{4}$, then either $\text{E}(\phi_{i,k}) > \frac{k}{8}$ or $\text{D}(k) > \frac{k}{8}$. We individually bound these probabilities first. To begin with, fix any historical path $\phi \in \mathcal{H}_k$ with disjoint tree errors. For any $l \geq 1, h \geq 1$, with $l + h - 1 \leq k$, let $\phi[l, l + h - 1]$ be any successive chunk of tree code errors in ϕ . By this, we mean that $\phi[l] = \phi[l + 1] = \dots = \phi[l + h - 1] = p_j$ for some $j \in [n]$ ($j = i$ is also allowed), and the party p_j made tree code errors in chunked rounds $l, l + 1, \dots, l + h - 1$. Since ϕ has disjoint tree errors, then p_j made no tree code errors in $l - 1$. Also, we either have $\phi[l + h + 1] \neq p_j$ or p_j makes no tree code error in iteration $l + h$. Now, since the tree code distance of $\mathcal{T}$ is $\frac{1}{2}$, a tree code error of length h implies at least $\frac{h}{4}$ symbols from the tree code output alphabet Σ were wrongly decoded by p_j in the chunked rounds l to $l + h - 1$ – see Lemma 4. Therefore, we can conclude that in every historical path $\phi \in \mathcal{H}_k$ with disjoint tree errors, there are at least $\frac{\text{E}(\phi)}{4}$ chunked rounds l , where $\phi[l]$ makes a decoding error in the **A2A** scheme in the second phase of communication.

Now, since $\log |\Sigma| = O(c)$ and each tree code symbol from Σ is communicated via $\log |\Sigma|$ rounds of **A2A** scheme, the probability of channel error, i.e., the probability of decoding a tree code symbol is $q = 2^{-\Omega(\sqrt{n})}$ by Theorem 3. Define the random variable X_l to be the indicator that $\phi[l]$ made a

channel error in iteration l . Then $\Pr(X_l = 1) = q$. Thus, we have

$$\begin{aligned}
\Pr\left(E(\phi) > \frac{k}{8}\right) &\leq \Pr\left(\sum_{l=1}^k X_l \geq \frac{k}{32}\right) \\
&= \sum_{s=\frac{k}{32}}^k \binom{k}{s} q^s (1-q)^{k-s} \\
&\leq 2^k q^{\frac{k}{32}} \\
&= 2^{-\Omega(k\sqrt{n})},
\end{aligned} \tag{5}$$

where the last equality uses $q = 2^{-\Omega(\sqrt{n})}$.

On the other hand, define Y_l to be the indicator random variable of some party making a decoding error during **SimChunk** for iteration l . Since, the communication in **SimChunk** consists of $c = O(1)$ rounds of **A2A**, we have by Theorem 3 that $r = \Pr(Y_l = 1) = 2^{-\Omega(\sqrt{n})}$. Therefore, we have

$$\begin{aligned}
\Pr\left(D(k) > \frac{k}{8}\right) &= \Pr\left(\sum_{l=1}^k Y_l \geq \frac{k}{8}\right) \\
&= \sum_{s=\frac{k}{8}}^k \binom{k}{s} r^s (1-r)^{k-s} \\
&\leq 2^k r^{\frac{k}{8}} \\
&= 2^{-\Omega(k\sqrt{n})},
\end{aligned} \tag{6}$$

where the last equality again uses $r = 2^{-\Omega(\sqrt{n})}$.

Now, noting that the total number of historical paths in $\mathcal{H}_k$ with disjoint tree errors is upper bounded by $n^k = 2^{k \log n}$, the probability that there exists $\phi_{i,k}$ satisfying $E(\phi_{i,k}) > \frac{k}{8}$ is upper bounded by $2^{-\Omega(k\sqrt{n})}$ using union bound and (5). This, together with the union bound and (6) upper bounds the probability of the existence of $\phi_{i,k}$ satisfying $E(\phi_{i,t}) + D(k) > \frac{k}{4}$ is $2^{-\Omega(\sqrt{n})}$. In other words, $\Pr(\text{RP}(p_i, k) < \frac{k}{2}) = 2^{-\Omega(k\sqrt{n})}$. Taking union bound over all $i \in [n]$, we have that

$$\Pr\left(\exists i \in [n] : \text{RP}(p_i, k) < \frac{k}{2}\right) = 2^{-\Omega(k\sqrt{n})}.$$

This completes the proof of Lemma 7.

C Analysis of the simultaneous simulation scheme

In this section we analyze the coding scheme in Algorithm 3 and prove Theorem 6.

We start with Appendix C.1, defining a notion of progress made by the simulation and analyzing it in light of error events that may happen during the simulation. Then, in Appendix C.2 we bound the probability that a complete simulation made insufficient progress.

C.1 Progress analysis of Algorithm 3

We define the progress made by the simulation protocol in Algorithm 3 at a given rounds r to be the number of rounds of the original protocol π *correctly* and *simultaneously* simulated so far. In other words, we define the progress at round r by

$$\text{RP}(r) \triangleq \max \left\{ 1 \leq k \leq r : k \text{ rounds have been correctly simultaneously simulated} \right\}, \quad (7)$$

meaning that (a) the $\text{RP}(r)$ first simulated rounds are correct, namely, $\{\text{Info}(\sigma_{i,j}(r)[1, \text{RP}(r)])\}_{i,j}$ is consistent with π ; and (b) the simulation is simultaneous, namely, for all i, j , and for all $\ell \leq \text{RP}(r)$, we have $\text{Rnd}(\sigma_{i,j}(r)[\ell]) = \text{Rnd}(\sigma_{1,2}(r)[\ell])$. Under this definition, the simulation is successful whenever $\text{RP}(2R) \geq R$.

In order to analyze the quantity $\text{RP}(r)$, we first define $\mathbf{1}_H(l)$, $\mathbf{1}_{BK}(l)$, and $\mathbf{1}_E(l)$ to be, respectively, the indicators of whether any party sent a H, a BK, and made a tree code decoding error in round l . Furthermore, we shall use notations of the form $\mathbf{1}_{A \cap B}(l)$ to denote the indicator of whether $A \cap B$ occurred in round l . For example, $\mathbf{1}_{E \cap BK}(l)$ is the indicator of the event that some party made a tree code decoding error and some other party sent a BK in round l .

We say that after round r a party i is *ahead* of a party j if party i has simulated (possibly incorrectly) a (strictly) larger number of rounds than party j . Alternatively, we shall say that party j is *behind* party i after round r . Note that if i is ahead of j after round r , then in round $r + 1$ we must have $\text{Ind}_i(r) > \text{Ind}_j(r)$. Furthermore, if party i did not make any tree code decoding error in round $r + 1$, then we also have $\text{Ind}_i(r) > \text{Ind}_{i,j}(r) = \text{Ind}_j(r)$.

We now relate the progress made by the parties till the last round, i.e., $\text{RP}(2R)$, with the total number of rounds with tree code decoding errors, i.e., $\sum_{l=1}^{2R} \mathbf{1}_E(l)$ through a series of technical lemmas. We will also use $\bar{E}$ to denote the complement of the event E .

Lemma 8. *For every iteration $r \leq 2R$, the following holds:*

$$r \leq \text{RP}(r) + \sum_{l=1}^r [\mathbf{1}_{BK}(l) + \mathbf{1}_E(l)]. \quad (8)$$

Proof. We shall prove this by induction. Note that the result trivially holds for $r = 0$. We now assume that the result holds till round $r - 1$. Now, note that $\sum_{l=1}^r \mathbf{1}_{BK}(l)$ and $\sum_{l=1}^r \mathbf{1}_E(l)$ are non-decreasing functions of r . Also, $\text{RP}(r)$ can go down by at most one at each iteration. Then, there are only two ways in which the induction hypothesis can be violated at r . The first possibility is when all three terms in the RHS of Equation (8) remaining unchanged; the other case is when the progress goes down by one while either there was no error or there was no back. We shall complete the proof by showing that both these cases cannot happen.

Consider the first case where $\text{RP}(r - 1) = \text{RP}(r)$ and $\mathbf{1}_{BK}(r) = \mathbf{1}_E(r) = 0$. In this case, since there were no tree code errors at iteration r , we have that for every $i \neq j$, $\hat{w}_{i,j}(r) = s_{j,i}(r - 1)$. As a result, the parties know exactly the iterations in which an information bit was sent and iterations where a BK or H was sent. As a result, the parsed transcripts at every party now contain correct iteration numbers. Since, progress did not improve in iteration r , then either some party was ahead of at least one party, or there was an issue in the consistency check. In either case, at least one party will send a BK which violates the fact that $\mathbf{1}_{BK}(r) = 0$. Hence, this case cannot occur.

The remaining case involves $\text{RP}(r) = \text{RP}(r-1) - 1$. In this case, some party must have sent a BK that caused deleting one round and decreasing the progress, hence, $\mathbf{1}_{\text{BK}}(r) = 1$. Therefore, to violate the induction hypothesis, it must be that $\mathbf{1}_{\text{E}}(r) = 0$. Now, as argued for the previous case, no tree code decoding error implies the parties know exactly the iterations in which an information bit was sent and iterations where a BK or H was sent. Again, in such a situation, the parties will either correctly simulate the next round if no one was ahead of the others and they were consistent, or the parties would either send a H (if they were consistent and behind the remaining parties) or a BK (if they were ahead of some party, or were inconsistent). Neither of these actions decreases progress, and hence $\text{RP}(r) \geq \text{RP}(r-1)$, which contradicts our assumption that $\text{RP}(r) = \text{RP}(r-1) - 1$. Therefore, this case is also vacuous, and hence Equation (8) holds by induction. $\square$

To proceed, we define the *gap* after iteration r , denoted by $\text{Gap}(r)$, as the maximum number of rounds incorrectly or non-simultaneously simulated by the parties. Formally,

$$\text{Gap}(r) \triangleq \max_{i \in [n]} \text{Ind}_i(r) - \text{RP}(r).^8$$

Therefore, $\text{Gap}(r) = 0$ implies that the parties are synchronized after iteration r , and every simulated round is correctly and simultaneously simulated. We shall use the convention that $\text{Gap}(0) = 0$.

The next lemma relates the number of iterations where a tree code decoding error was made with the number of iterations where a BK was sent and no errors were made.

Lemma 9. $\forall r \in [2R]$, it holds that

$$\sum_{l=1}^r \mathbf{1}_{\text{BK} \cap \overline{\text{E}}}(l) \leq 2 \sum_{l=1}^r \mathbf{1}_{\text{E}}(l). \quad (9)$$

Proof. First, suppose that some party makes a tree-code error in iteration $r+1$. Then the gap between the parties can increase by at most two, and thus $\text{Gap}(r+1) \leq \text{Gap}(r) + 2$. To see this, note that the maximum increase in gap occurs when a party maximizing $\text{Ind}_i(r)$ decides to transmit information symbols in round $r+1$ (i.e., executes line 14) while, a party minimizing $\text{Ind}_i(r)$ sends a BK which deletes overall progress.

Alternately, assume no tree-code errors were made in iteration $r+1$. If $\text{Gap}(r) = 0$, then all parties were consistent. Therefore, they correctly simulate the next round (execute line 14) ensuring $\text{Gap}(r+1) = \text{Gap}(r) = 0$. On the other hand, if $\text{Gap}(r) > 0$, then by the simulate-hold-rewind mechanism (lines 9, 16 and 22), the parties which are ahead of at least one other party send BK and the remaining parties send either a H (if it was consistent) or a BK (due to inconsistency). In both cases, the progress remains unchanged, i.e., $\text{RP}(r) = \text{RP}(r+1)$, but the maximum index reduces by one, i.e., $\max_{i \in [n]} \text{Ind}_i(r+1) = \max_{i \in [n]} \text{Ind}_i(r) - 1$. Hence, the gap reduces exactly by one.

To summarize the above, we have that if $\mathbf{1}_{\text{E}}(r) = 1$, then $\text{Gap}(r+1) \leq \text{Gap}(r) + 2$, and if $\mathbf{1}_{\text{BK} \cap \overline{\text{E}}}(r) = 1$, then $\text{Gap}(r+1) = \text{Gap}(r) - 1$. Otherwise, $\text{Gap}(r+1) = \text{Gap}(r)$. Since, initially $\text{Gap}(0) = 0$, we have

$$0 \leq \text{Gap}(r+1) \leq 2 \sum_{l=1}^r \mathbf{1}_{\text{E}}(l) - \sum_{l=1}^r \mathbf{1}_{\text{BK} \cap \overline{\text{E}}}(l),$$

which completes the proof. $\square$

⁸ Recall that $\text{Gap}(r)$ is the gap at the end of iteration r , however, since it is being computed according to line 7, it does not change throughout the iteration. Communication done in iteration r can only affects $\text{Gap}(r+1)$.

The following lemma relates progress made by the protocol to the number of rounds where at least one BK was sent.

Lemma 10. $\forall r \in [2R]$,

$$RP(r) \leq r - \sum_{l=1}^r \mathbf{1}_{BK}(l), \quad (10)$$

Proof. We prove this by induction. The base case of $r = 1$ is trivial as in the first round every party sends information bits, and hence no BK is sent. Thus, the right-hand side equals 1, while the left-hand side cannot surpass 1. Now, assume that the statement is true upto round $r - 1$, i.e., we have $RP(r - 1) \leq r - 1 + \sum_{l=1}^{r-1} \mathbf{1}_{BK}(l)$. Then, the statement of Lemma 10 is violated in round r only if some party sent BK in round r and $RP(r) = RP(r - 1) + 1$. This situation, however, is impossible as progress cannot occur unless all parties simultaneously send an information bit; the fact that a party sent a BK immediately implies that there is no simultaneous simulation and thus no progress, in iteration r . The statement then follows. $\square$

Lemma 11. *For every iteration $r \leq 2R$, we have*

$$\sum_{l=1}^r \mathbf{1}_E(l) \geq \frac{r - RP(r)}{5}. \quad (11)$$

Proof. Using Lemma 8, we have

$$\begin{aligned} 2r &\leq 2RP(r) + 2 \sum_{l=1}^r \mathbf{1}_{BK}(l) + 2 \sum_{l=1}^r \mathbf{1}_E(l) \\ &\stackrel{(a)}{\leq} RP(r) + r + \sum_{l=1}^r \mathbf{1}_{BK}(l) + 2 \sum_{l=1}^r \mathbf{1}_E(l), \end{aligned}$$

where (a) is due to Lemma 10. Therefore, we have

$$\begin{aligned} r &\leq RP(r) + \sum_{l=1}^r \mathbf{1}_{BK}(l) + 2 \sum_{l=1}^r \mathbf{1}_E(l) \\ &\leq RP(r) + \sum_{l=1}^r \mathbf{1}_{BK \cap E}(l) + \sum_{l=1}^r \mathbf{1}_{BK \cap \bar{E}}(l) + 2 \sum_{l=1}^r \mathbf{1}_E(l) \\ &\leq RP(r) + 5 \sum_{l=1}^r \mathbf{1}_E(l), \end{aligned}$$

where the last inequality follows using Lemma 9 and noting that $\sum_{l=1}^r \mathbf{1}_{BK \cap E} \leq \sum_{l=1}^r \mathbf{1}_E(l)$. $\square$

Lemma 11 relates the number of iterations with tree code decoding errors till iteration r and the progress made so far. To complete the proof of Theorem 6, it remains to suitably bound the probability of error in Algorithm 3, by bounding the number of iterations with tree code decoding errors. This is dealt with in the following section.

C.2 Success Probability of Algorithm 3

In this section, we bound the probability of error of the simultaneous simulation scheme in Algorithm 3. Recall that the simulation fails if either it returns incorrect transcripts of the original protocol π , or if certain rounds of π were not simultaneously simulated. Let $\mathcal{E}_1$ denote the event that $\text{RP}(2R) < R$, i.e., that the simulation did not correctly and simultaneously simulate all π 's rounds during the first $2R$ iterations. Further, let $\mathcal{E}_2$ denote the event that some party made a tree code decoding error of length larger than R after the completion of $3R$ rounds. We now bound the probability of these two events, since if both do not happen, the simulation succeeds.

Overview. We begin our analysis by providing a bound on the number of A2A errors required in the simulation protocol of Algorithm 3 to ensure that $\mathcal{E}_1$ occurs. To do so, we first claim that for $\mathcal{E}_1$ to occur, out of the set of $2R$ iterations, one can find disjoint sets of consecutive iterations, referred to as bad runs (precisely defined later), where one party makes tree code decoding errors in all of those iterations. The total number of iterations taking part in these bad runs is shown to be at least $\frac{R}{5}$ given $\mathcal{E}_1$ occurs. By Lemma 4, in each bad run, the party making the tree code decoding errors must make A2A decoding errors in at least one-fourth of the iterations involved in that particular bad run. We bound this probability which then gives us a bound on $\Pr(\mathcal{E}_1)$. To complete the proof of Theorem 6, we then bound $\Pr(\mathcal{E}_2)$ using Lemma 4 and the probability of A2A errors.

Notations. Before we proceed, we set up some notation. Assume that party p_i makes a tree code error in iteration r . Recall that in any round, p_i decodes one tree-coded string from each of its neighbours. Hence, a tree-code error in iteration r can stem from incorrectly decoding one or more of these $n - 1$ tree-coded strings. Now, choose the string for which p_i made the tree code decoding error of maximum length, and denote this length by t . Then we say that p_i has a *bad suffix* of length t at iteration r .

Next, we define a *run* $I \subseteq [2R]$ to be a set of *consecutive* iterations. The run is said to start at iteration $\min I$ and end at iteration $\max I$; the *length* of the run, $|I| = 1 + \max I - \min I$ is the cardinality of the set I . Finally, a run I is said to be a *bad-run* if there exists some $j \in [n]$ such that p_j has a bad suffix of length $|I|$ at iteration $\max I$ where the run ends.

A bound on $\Pr(\mathcal{E}_1)$. We start with a technical lemma showing that if $\mathcal{E}_1$ occurs, we can lower bound the total length of some disjoint bad-runs.

Lemma 12. *If $\mathcal{E}_1$ occurs, then there exists a disjoint set of $K \geq 1$ bad-runs $I_1, I_2, \dots, I_K$ satisfying $\sum_{k=1}^K |I_k| \geq \frac{R}{5}$. Furthermore, if iteration $l \notin \bigcup_{k=1}^K I_k$ then $\mathbf{1}_E(l) = 0$, i.e., no party makes a tree code decoding error in iteration l .*

Proof. We construct the bad runs $I_1, \dots, I_K$ starting from the last iteration as follows. Let $r \leq 2R$ be the last iteration where some party made a tree code decoding error. Among the parties that made a tree code decoding error at iteration r , let p_i be the party with the longest bad suffix at iteration r . Let the length of this bad suffix be t . Then, define $I_1 \triangleq \{r - t + 1, \dots, r\}$, and observe that I_1 is indeed a bad run.

To construct the remaining bad runs $I_2, \dots, I_K$ follow the same procedure starting from iteration $r - t$. This process terminates when either all iterations have been used up, or we reach some iteration r' such that $\mathbf{1}_E(l) = 0$ for every $l \leq r'$.

Note that the bad runs $I_1, \dots, I_K$ constructed in this manner are indeed disjoint, and for every iteration $l \notin \cup_{k=1}^K I_k$, we have $\mathbf{1}_E(l) = 0$. It remains to show that $\sum_{l=1}^k |I_l| \geq \frac{R}{5}$, conditioned on the occurrence of $\mathcal{E}_1$.

To see this, since $\{r \in [2R] \mid \mathbf{1}_E(r) = 1\} \subseteq \bigcup_{l=1}^k I_l$, Lemma 11 then gives

$$\sum_{l=1}^k |I_l| \geq \sum_{r=1}^{2R} \mathbf{1}_E(r) \geq \frac{2R - \text{RP}(R)}{5}.$$

If $\mathcal{E}_1$ occurs we have $\text{RP}(2R) < R$, yielding,

$$\sum_{l=1}^k |I_l| \geq \frac{R}{5}.$$

□

With the help of Lemma 12 we can show that if the simulation did not succeed, this must be due to many iteration in which the A2A scheme failed to provide a resilient exchange of information.

Lemma 13. *Given that $\mathcal{E}_1$ has occurred, there exists a set $I \subseteq [2R]$ of length $|I| \geq \frac{R}{20}$, such that there is an A2A error in every iteration $r \in I$.*

Proof. Consider the bad-runs $I_1, \dots, I_K$ given by Lemma 12. For any $k \in [K]$ let $M_k = \max I_k$. Then, by construction, some party p_i has a bad suffix of length $|I_k|$ at iteration M_k . Since the tree code has distance $\alpha = \frac{1}{2}$, Lemma 4 says that there must be a subset $J_k \subseteq I_k$ satisfying $|J_k| \geq \frac{1}{4}|I_k|$, such that p_i made an A2A decoding error for every round in J_k . The result then follows by defining $I = \cup_{k=1}^K J_k$, and noting that Lemma 12 implies that $|I| \geq \frac{1}{4} \sum_{k=1}^K |I_k| \geq \frac{R}{20}$. □

Since a decoding error in A2A has a small probability of $2^{-\Omega(\sqrt{n})}$ (Theorem 4), the existence of many iterations with decoding errors in A2A also has a small probability, which in turn bounds the probability of the occurrence of the event $\mathcal{E}_1$.

Lemma 14.

$$\Pr(\mathcal{E}_1) \leq 2^{-\Omega(R\sqrt{n})}. \quad (12)$$

Proof. Due to Lemma 13, to bound $\Pr(\mathcal{E}_1)$, we can bound the probability to have $R/20$ iterations with A2A decoding error. Recall from Theorem 4 that the probability of an A2A decoding error in any iteration is $2^{-\Omega(\sqrt{n})}$, and since the noise is independent, the failure probabilities of different A2A instances (i.e., different iterations) are independent. Then, probability that there exists a set of iterations $I \subseteq [2R]$ of length $|I| \geq \frac{R}{20}$ such that some party makes an A2A decoding error in iteration $l \in I$, is upper bounded by

$$\sum_{k=\frac{R}{20}}^{2R} \binom{2R}{k} 2^{-k\Omega(\sqrt{n})} \leq \sum_{k=\frac{R}{20}}^{2R} \binom{2R}{k} 2^{-R\Omega(\sqrt{n})} \leq 2^{2R} 2^{-R\Omega(\sqrt{n})} = 2^{-R\Omega(\sqrt{n})}. \quad (13)$$

Then, by Lemma 13, we have $\Pr(\mathcal{E}_1) \leq 2^{-R\Omega(\sqrt{n})}$. □

A bound on $\Pr(\mathcal{E}_2)$. To complete the error probability analysis, we now bound the probability of the event $\mathcal{E}_2$.

Lemma 15.

$$\Pr(\mathcal{E}_2) \leq 2^{-\Omega(R\sqrt{n})}. \quad (14)$$

Proof. To incorrectly decode the first $2R$ symbols of the $3R$ -long tree-coded string, a party must have a bad suffix of length at least $R + 1$ after $3R$ iterations.

Recall that the tree code $\mathcal{T}$ has a distance $\alpha = \frac{1}{2}$. Thus, by Lemma 4, a party having a bad suffix of length ℓ must have made at least $\frac{\ell}{4}$ A2A decoding errors in the last ℓ iterations. Moreover, since the A2A decoding error event includes any one of the n parties making an error, the event that there exists a party with a bad suffix of length ℓ implies that at least $\frac{\ell}{4}$ of the last ℓ iterations had A2A decoding errors.

Theorem 4 shows that the probability of an A2A decoding error is $2^{-\Omega(\sqrt{n})}$. Then, the probability that there exists a party with a bad suffix of length $R + 1$ at the end of Algorithm 3 can be upper bounded by

$$\sum_{\ell=R+1}^{3R} \binom{\ell}{\frac{\ell}{4}} 2^{-\ell\Omega(\sqrt{n})} \leq \sum_{\ell=R+1}^{3R} 2^\ell 2^{-R\Omega(\sqrt{n})} \leq 2R 2^{3R} 2^{-R\Omega(\sqrt{n})} \leq 2^{-R\Omega(\sqrt{n})}.$$

Therefore,

$$\Pr(\mathcal{E}_2) \leq 2^{-R\Omega(\sqrt{n})}. \quad \square$$

Completing the proof of Theorem 6. We now combine all the technical results so far to complete the proof of Theorem 6.

Proof of Theorem 6. Since Algorithm 3 takes $3R = 3|\pi|$ iterations, where each iteration executes exactly a single A2A instance, which in turn takes $O(1)$ rounds of communication, and no further communication occurs, the simulation takes $3R \times O(1) = O(|\pi|)$ rounds of communication.

Following Lemma 14 and Lemma 15, the failure probability of the simulation can be upper bounded by

$$\Pr(\text{error in Alg 3}) \leq \Pr(\mathcal{E}_1) + \Pr(\mathcal{E}_2) \leq 2 \cdot 2^{-\Omega(R\sqrt{n})} = 2^{-\Omega(R\sqrt{n})},$$

This completes the proof. $\square$

D Merged IC: A short overview

The IC scheme stated in Theorem 7 merges the chunked nature of ChunkRS with the capability of simultaneous simulation. A chunk size $c = O(1)$ is considered, and the protocol simulation proceeds in chunks of c rounds. The protocol alphabet is thus chosen as $S = \{\{0, 1\}^c, \text{BK}, \text{H}\}$, and we shall use a tree code $\mathcal{T} : S^* \rightarrow \Sigma^*$ with distance $\alpha = \frac{1}{2}$ to protect the transmissions.

As in the two previous schemes, all parties maintain their sent and received transcripts $(s_{i,j}, w_{i,j})_{j \neq i}$. Then, at the beginning of any chunk, party i first decodes all the symbols received from party j in the previous chunks to obtain the string $\hat{w}_{i,j}$. Then, the parties parse these strings according to the

parsing scheme used in the simultaneous simulation scheme, but in a chunk-by-chunk instead of a round-by-round fashion. This is then followed by a consistency check to determine whether a party believes that the simulation so far has been consistent and simultaneous.

Following this, the parties begin the simulation of the next chunk as in ChunkRS. As in ChunkRS, if the parties failed the consistency check, they send dummy 0s throughout the simulation of the next chunk and send a BK to the remaining parties after the simulation. However, the key difference with the simulation of the next chunk in ChunkRS and the merged IC lies in the situation where the party believes that no party is behind it, but some parties are strictly ahead of it. In such a situation, in ChunkRS, the party would have continued with the simulation and then sent the simulated chunk to the remaining parties. However, in the merged IC, even in this scenario, the party will start sending dummy 0s during the chunk simulation and then send a H to the remaining parties at the end of the chunk simulation.

If the original (noiseless) protocol had R rounds, then this process continues for $\frac{2R}{c}$ chunked rounds. Following this, to avoid the decoding error at the end which can hamper simultaneous simulation, every party sends additional R zeros to the remaining parties. Then, every party makes a fresh tree code decoding of the symbols received thus far, truncates them to a size $\frac{2R}{c}$, and then returns the relevant parsed strings truncated to R bits.

The proof technique is similar to that of the scheme with simultaneous simulation. We analyze the progress made by the parties and relate it to the number of chunked rounds with tree code decoding errors. Next, we show that for the global progress to be lower than $\frac{R}{c}$ even after the $\frac{2R}{c}$ chunked round, there exists a large number of chunked rounds where at least one party made a tree code decoding error. Using this, we shall be able to show that there exists a history where a large number of A2A decoding errors were made, and this would be a low-probability event. As a result, we shall be able to show that the global progress must have been large enough with high probability. The additional dummy rounds ensure that all parties are able to successfully return the correct transcripts unless there is a tree code error of large length, which again is a low probability event. To avoid repetitive proofs, we omit the details.

E MPC Proofs

E.1 Proof of Lemma 1

Proof. We prove this by inducting over t .

- Base case $t = 0$ corresponds to the simulator and the protocol views being statistically indistinguishable after the initialization phase. In this phase, the view of all corrupt parties in a real protocol execution consists of $|C|$ shares of each party's input, encoded with an ECC of size $O(\kappa)$. Since we use the standard (n, n) -additive secret sharing, $\leq n - 1$ shares of any two secrets are perfectly indistinguishable. Hence, the protocol view is identical to that of the simulated view before the start of the evaluation phase.
- Let the statement hold for any $t = k - 1$ for $0 < k < |\pi_f|/3$. We further show it holds for $t = k$.
- $\widetilde{\text{View}}_{\Pi}^{\text{BSC}}(C, x, k)$ includes $\widetilde{\text{View}}_{\Pi}^{\text{BSC}}(C, x, k - 1)$ and the messages sent during execution of k^{th} chunk. In this k^{th} chunk of the protocol, the parties execute an A2A-NAND gate (say g , with real or dummy inputs), then exchange tree code symbols via A2A. The simulator generates the following view for the NAND execution in the k^{th} chunk of the protocol:

- For each party P_i it uses the variable rcvMsg_i to compute next gate g_i to be evaluated (using nextNANDgate) and the corresponding inputs (a_i, b_i) (using nextNANDinp only for honest parties). By induction, rcvMsg_i is indistinguishable from the received messages by party P_i in the protocol. This implies the NAND inputs generated by nextNANDinp are indistinguishable for both the protocol and the simulator.
 - By construction, each NAND execution uses independent ROT correlations. Hence, the A2A-NAND protocol view in k^{th} chunk is perfectly indistinguishable from (i) the output of simulator in Theorem 8 with inputs $\{a_i, b_i\}_{i \in C}$, and also from (ii) the output of $\text{Sim}_{\text{NAND}}^{\text{BSC}}$ simulator from Theorem 9 with same inputs and any arbitrary NAND output.
- Both the protocol and the simulator have statistically indistinguishable distributions for rcvMsg_i until the NAND execution in the k^{th} iteration. The function nextSymbol is a deterministic function of each party's rcvMsg_i . Hence, the simulator can generate the tree symbols input to the A2A protocol as described in Step (v) - indistinguishable from the A2A tree symbols sent in the protocol.

□